Second-year Internship Report

Visual-Inertial Data Collection Platform and Sensor Fusion Benchmarking

CRTA, Zagreb, Croatia

May / August 2026

Seatech Major SYSMER

CLAVERIE Timm

Tutor : Dr. ŠVACO Marko

Supervisor : ĆARAN Branimir

I would like to thank first Branimir ĆARAN, who guided me, advised me, gave me his time, and helped me develop my thinking as a systems engineer in the field of embedded robotics; and I would also like to thank the laboratory director, Dr. Marko ŠVACO, for placing his trust in me and give me the opportunity of the internship.

Abstract

To use drones outdoors, it is essential to know their exact flight path. Currently, the CRTA laboratory uses a wide variety of drones capable of carrying payloads. Although various position estimation systems exist, they have drawbacks—particularly GPS—because what would happen if the signal between the drone and the GPS were to be jammed?

The goal of the project is to build a modular sensor platform capable of estimating the platform's position using odometry and data fusion packages. To achieve this, we must design a ROS2 architecture on an embedded mini-computer, control the sensors by calibrating them, and conduct a large number of tests to refine the parameters of our algorithms.

To ensure that this work is immediately usable by the lab's researchers, we developed a ROS2 pipeline capable of recording and exporting 6D coordinates to a processing computer. And, to verify that our system was truly reliable, we compared our measurements with those from OptiTrack during several practical tests.

Key Words : GPS-denied, Visual and inertial data acquisition platform, Sensor Fusion, ROS2, middleware architecture, synchronization algorithms, network configuration, IMU sensors, optical data streams, quantitative analysis.

Regarding scientific and academic integrity, you can find my pledge against plagiarism here 8.2.

Contents

1 Introduction **4**
1.1 Project Management and Organization . . . 5
1.1.1 Work Planning . . . 5
1.1.2 Communication Tools . . . 5
1.2 Functional Analysis . . . 5
1.2.1 Requirement Expression (Bull Chart) . . . 5
1.2.2 Interactions and Constraints . . . 6
1.3 State of the Art . . . 7
1.3.1 Visual-Inertial Odometry . . . 7
1.3.2 Merging Algorithms . . . 7
1.4 Experimental Approach of the Project . . . 8

2 Immersion at CRTA **9**
2.1 The Laboratory and How It Operates . . . 9
2.1.1 Core Missions . . . 9
2.1.2 Infrastructure Organization . . . 10
2.1.3 Service Organization . . . 10
2.2 Work Organization . . . 10
2.2.1 Commercial Policies . . . 11
2.2.2 Suppliers and Competitors . . . 11
2.2.3 Key Contacts . . . 11
2.2.4 Stakeholders . . . 11
2.3 Corporate Social Responsibility . . . 12
2.4 Preliminary Task: ROS2 Navigation and Path Following . . . 12

3 Overview of the Equipment **14**
3.1 Operating Mode . . . 14
3.1.1 Computer . . . 14
3.1.2 IMU . . . 15
3.1.3 Cameras . . . 16
3.1.4 Holybro . . . 17
3.2 Calibration . . . 18
3.3 Limitations . . . 18

4 CAD Modeling of the Sensor Platform **19**
4.1 Flexible Experimentation Platform . . . 19
4.2 Drone Platform . . . 21

5 Workspace Architecture **23**
5.1 Overall Architecture Adjustments . . . 23
5.2 Data Acquisition Chain . . . 23
5.2.1 Sensor Holybro . . . 23
5.2.2 Sensor Camera . . . 25
5.2.3 Sensor IMU . . . 26
5.3 VIO Odometry . . . 27
5.3.1 Control of input and output flows . . . 27
5.3.2 OpenVins Calibration . . . 28
5.3.3 Quality Assessment . . . 28
5.4 Sensor Fusion . . . 31
5.4.1 UKF . . . 32
5.4.2 EKF . . . 33

6 Experimentations and Analysis **34**
6.1 Gradual Sensor Fusion . . . 34
6.1.1 VIO + Holybro . . . 34
6.1.2 VIO + OlixSense . . . 34
6.1.3 VIO + Holybro + OlixSense . . . 35
6.1.4 Summary . . . 35
6.2 Initial Dynamic Test . . . 35
6.3 Complex Trajectories . . . 35
6.4 Robustness and Blind Tracking . . . 36
6.4.1 Summary . . . 37
6.5 Platform Mounted on Spot . . . 37

7 Conclusion **38**
7.1 Compliance with Specifications . . . 39
7.2 Improvements . . . 39
7.3 Opening . . . 40

Bibliographie **42**

8 Annexe **44**
8.1 GitHub . . . 44
8.2 Internship Informations . . . 44
8.3 Project Management and Functional Analysis . . . 47
8.4 Tree structure . . . 48
8.5 Overview of Equipment . . . 49
8.5.1 Connection to Jetson . . . 49
8.5.2 Display Olive Robotics Sensors . . . 49
8.5.3 Getting Started H-Flow . . . 50
8.5.4 Sensors Calibration . . . 54
8.6 Overall Rules . . . 58
8.6.1 Communication and ID Rules . . . 58
8.6.2 USB Rules . . . 59

8.7 Workspace Architecture 60
8.7.1 OlixVision Data Acquisition 60
8.8 Common Files and Commands 61
8.8.1 Image decompressor 61
8.8.2 ROS2-to-ROS1 bag 61
8.8.3 Tf Tree 61
8.9 OptiTrack Tutorial 62
8.10 Package Evo Tutorial 63
8.11 Data Acquisition Tutorial 65

1 Introduction

The vast majority of drones use GPS to estimate their position in real time. However, in the event of GPS jamming, the position is lost and it becomes impossible to assess whether a mission was successful. It must be possible to accurately reconstruct the flight path. It is therefore essential that the drone be capable of estimating its position and orientation completely autonomously.

This project proposes a modular platform design to evaluate the effect of integrating different sensors. The goal is to determine a robot's position using data fusion algorithms to refine a pose previously estimated by a visual and inertial pose estimator, all structured by a ROS architecture intended for deployment on the drones at the [1] laboratory.

The project is divided into three areas:

- **Modeling:** design and assemble various platforms for testing.
- **Integration :** build a ROS2 architecture, calibrate sensors, and integrate pose estimation algorithms.
- **Evaluation :** quantitatively analyze all results saved in datasets, then deliver a documented ROS2 architecture that can be used by researchers at the CRTA laboratory.

The system's performance will be evaluated by comparing our results to those of OptiTrack during testing.

The system is organized in such a way that trajectory evaluation is possible via post-processing using recorded datasets. To prevent the user from having to interact with the code, the system is nearly autonomous, and all necessary steps have been documented.

The code is available on GitHub at the following page: `https://github.com/timm-clv/SensorFusion`. It is publicly available and includes a ReadMe file offering alternative methods for evaluating drone pose for various research laboratories and other users.

1.1 Project Management and Organization

To successfully complete this project, I had to establish a rigorous organizational structure. The goal was to coordinate my work tasks over several weeks while managing access to equipment (drones, 3D printers) and facilities (OptiTrack area).

1.1.1 Work Planning

Since this was an individual project, no Gantt chart was created. I organized my work myself to : set deadlines for each major milestone ; prioritize tasks ; identify dependencies. For example, camera calibration must be completed before starting position estimation.

1.1.2 Communication Tools

- A shared, publicly accessible GitHub repository with a history of changes to prevent overwriting code
- Emails and in-person meetings to discuss, request equipment, and conduct tests

1.2 Functional Analysis

The purpose of the functional analysis is to provide a framework for our development and to translate the expectations of the CRTA laboratory into technical functions and constraints that our system must meet. We are basing this on the requirements specification we have drafted; see here.

1.2.1 Requirement Expression (Bull Chart)

To determine the need that our project addresses, we modeled the situation using a bull chart (see 1.1).

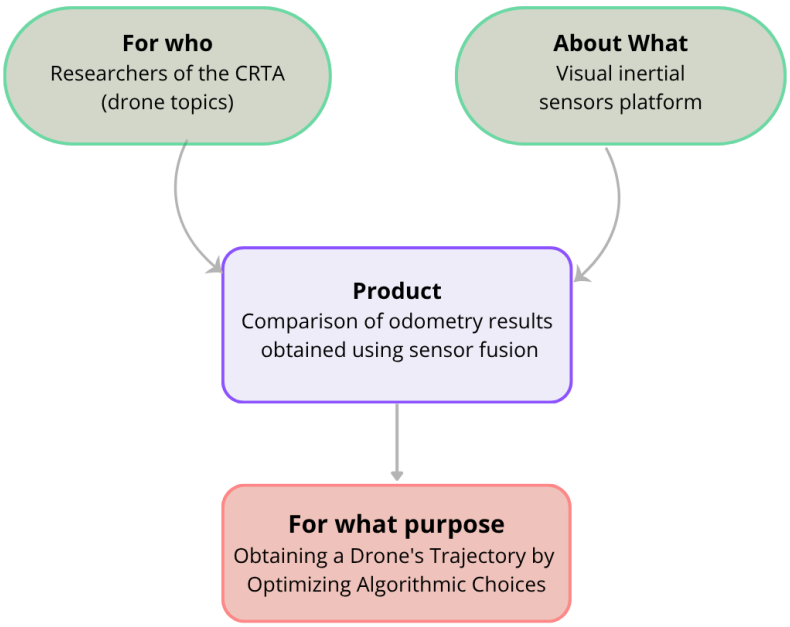


Figure 1.1: Bull chart diagram

The deliverable is a ROS2 architecture that must provide on-board visual-inertial data acquisition capable of delivering reliable data for autonomous navigation. The goal is to evaluate visual-inertial odometry (VIO) and sensor fusion algorithms against ground truth data provided by OptiTrack. A sensor platform must be designed to bridge the gap between the physical environment and the localization algorithms.

1.2.2 Interactions and Constraints

To analyze the constraints, we have identified the system's interactions using an Octopus diagram (see 8.3). This diagram allows us to define the Functional Specifications (CDCF), summarized in Table 1.1.

These specifications highlight one Main Function (FP1) and five Constraint Functions (FC). FP1 expresses the requirement, and the constraint functions define the scope of development.

Ref.	**Function**	**Evaluation Criteria**	**Expected Level**
FP1	Provide the 6D position of the platform to the user	Position, orientation, topics synchronisations and ease of acquisition	Quick 6D analysis
FC1	Provide a functional and documented ROS2 architecture	Deliverables provided	Code and README on GitHub
FC2	Design platforms	Support all sensors and be versatile	Be flexible to changes
FC3	Use the provided sensors	Sensor control	Controlled processing of data streams
FC4	Record and analyze multiple tests	Easy recording of rosbags and quantitative analysis	10 rosbags
FC5	Comparison against real-world data	measured position error	< 10 cm
FC6	Implement VIO and fusion algorithms	Multiple algorithms	2 of each

Table 1.1: Functional Specifications (CDCF)

In addition, we note that the main objective was to integrate the platform onto this drone (see 1.2), but there were only two of them, and both crashed during testing.

Figure 1.2: Initial drone to implement the sensor architecture

1.3 State of the Art

Reconstructing a drone's trajectory using sensors and algorithms has been extensively documented. The main challenge lies in the choice of sensors; however, once these have been selected, it is necessary to choose the right tools to obtain an initial pose estimate to be fused with other data. In the literature, two families of methods stand out for each of these steps

1.3.1 Visual-Inertial Odometry

The first step is to obtain an odometry track from visual and inertial data :

- SLAM (Simultaneous Localization and Mapping): enables an autonomous system to build a map of an unknown environment while simultaneously estimating its own position within that map. Example: *Isaac_ros_visual_slam* [9]
- VIO (Visual-Inertial Odometry): estimates the position and orientation (pose) of a device by fusing data from a camera and an inertial measurement unit (IMU). Example: *OpenVins* [8]

Since mapping is secondary to the topic at hand, we will implement VIO first.

1.3.2 Merging Algorithms

The second step is to improve the pose using a fusion filter:

- EKF (Extended Kalman Filter), which is the most common fusion filter in robotics.
- UKF (Sigma-Point Nonlinear Kalman Filter), which is better suited for systems with high dynamic drifts, such as flying drones.
- InEKF (Invariant Extended Kalman Filter), which exploits Lie group symmetries to be more robust than EKFs.
- MSCKF (Multi-State Constraint Kalman Filter), which is exactly the same filter used in VIOs.

1.4 Experimental Approach of the Project

Based on this state of the art and the constraints set forth in our Functional Specifications, there are multiple solutions for our system. To meet the needs of the CRTA laboratory, we have structured our scientific approach into four main stages:

1. **Evaluation of the Provided Sensors (Chapter 3):** Before choosing a solution, we will examine and test the various sensors to determine which ones are usable.

2. **Design of Various Sensor Platforms (Chapter 4):** Once the sensors have been selected, we will design several sensor platforms tailored to the CRTA's requirements.

3. **Construction of the ROS2 Workspace (Chapter 5):** The entire system will then be controlled via a ROS2 architecture combining *C++, Python, URDF, and YAML* files, requiring various stages of intrinsic and extrinsic calibration, as well as the implementation of packages for VIO and fusion algorithms. For the computer science portion, there are 6 design stages detailed in this tree structure 8.4.

4. **Tests, Analyses, and Validations (Chapter 6):** A multitude of tests will be conducted to adjust the various parameters of the ROS architecture, then to qualitatively evaluate the pose estimates, and finally to quantitatively validate the entire system.

You can find here a more detailed outline of the tasks performed.

2 Immersion at CRTA

The CRTA (Regional Center of Excellence for Robotics Technology) at FSB University in Zagreb is a laboratory affiliated with the Faculty of Mechanical and Naval Engineering. Officially opened in 2021, the center is intended to serve as a laboratory for students and researchers working in the fields of industrial, medical, and humanoid robotics [1].

2.1 The Laboratory and How It Operates

2.1.1 Core Missions

The center is structured around three research areas: medical robotics, industrial robotics, and autonomous systems coupled with artificial intelligence.

The CRTA's mission is to train Master's and PhD students on research topics. The center appears to serve as a bridge between the academic system and industry or research. In today's world, engineers who have already published scientific papers are highly sought after. Laboratories such as the one where the internship took place thus become excellent training centers.

The center offers a wide range of activities, including various projects for researchers and students, such as:

- a medical robot for brain surgery (trepanation).
- a SCARA robot capable of performing digital rectal exams and using a neural network to detect suspected prostate cancer.
- an image processing system for darts.
- a drone for reconnaissance and tracking individuals.
- a propeller-driven drone capable of vertical takeoff.
- detection of plastic objects using a camera system for a company.
- control of SCARA manipulator arms intended for industrial or medical use.
- a drone capable of climbing walls.
- autonomous drones for circuit racing and terrain mapping.
- drone motor control.
- a drone landing system on a moving object.
- an overtaking system for mobile racing drones.

2.1.2 Infrastructure Organization

The center has several departments. There are several rooms for students to conduct mechatronics exercises. There are also spaces for researchers, such as a medical laboratory. They have a simplified replica of an operating room equipped with robotic arms designed to perform trepanation or prostate analysis.
There are numerous test bench areas for robotic arms, conveyor belt assembly lines, 3D printers of all types, a workshop, and an open area for robots to conduct tests involving mapping, maze construction, or motion capture using OptiTrack.

Figure 2.1: Picture of the laboratory

2.1.3 Service Organization

The CRTA staff is divided into several groups: the administrative and financial department, professors with doctoral degrees, doctoral students and assistant professors, early-career researchers, and finally student assistants. This department is organized around:

- Research & Development : scientific publications by researchers, doctoral students, and students.
- Technology Transfer : studies conducted for other laboratories, companies, or schools.
- Education and Outreach : Academic training (from bachelor's to doctoral degrees) and outreach to schools to promote a culture of technology.

2.2 Work Organization

Spontaneity is strongly encouraged. You can come to work in the lab at any time of day and on any day of the week.

Ph.D. students and early-career researchers are involved in practical applications and industrial research.

The work is organized into research projects (e.g., INSPIRATION, MARINERO, ROBOCAMP) led by dedicated project managers, who may also be students—for example, in the case of robotics competitions.

There is strong synergy among researchers, mechanical engineers, robotics engineers, electronics engineers, and medical students. Everyone works in the same lab, and knowledge is shared quickly.

2.2.1 Commercial Policies

- Institutional funding : Securing European funds (European Regional Development Fund—ERDF, National Recovery and Resilience Plan).
- Partnerships : Joint technology development with major industrial groups, for example: optimizing battery cell production with the BMW Group.
- Support for technological transition : A business strategy focused on supporting companies in their green and digital transitions

2.2.2 Suppliers and Competitors

- Technology providers : Manufacturers of cutting-edge robotic equipment (use of KUKA robots for neurosurgical research), suppliers of sensors, AI components (e.g., NVIDIA), and software frameworks.
- Competitors : There are no competitors in the sense of an open market for a research laboratory, as they do not capitalize on their scientific publications. However, the CRTA operates in the same field as other European regional centers of excellence in robotics and AI, cutting-edge technology research institutes, and private R&D laboratories.

2.2.3 Key Contacts

- Prof. Dr. Bojan Jerbić : Professor, founder, and director of CRTA; a pioneer in robotics and AI.
- Assoc. Prof. Dr. Marko Švaco : Principal investigator leading applied projects (tourism, industry) and my mentor during my internship.
- Branimir Ćaran : Teaching assistant and doctoral student who served as my supervisor during the internship.
- Antonija Lelas: Administrative and financial project manager who handled my internship paperwork.

2.2.4 Stakeholders

- The internal stakeholders directly involved are employees of the University of Zagreb (FSB) and the Croatian university network (DATACROSS project).
- External stakeholders affected by the CRTA's activities include institutional entities, such as the European Union and the Croatian Ministry of Science, as well as industrial and private entities, such as the BMW Group, NOA Group, and Marina Punat.

2.3 Corporate Social Responsibility

- Medical and societal impact: Contributions to public health through the development of neurosurgical robotics.
- Sustainability and ecology: Leading projects that promote the circular economy and ecotourism (autonomous assistive robots, electric vehicles, marina cleanup).
- Accessibility and inclusion : Providing free educational resources (Student Corner) and opening the center to inspire younger generations to explore science.

2.4 Preliminary Task: ROS2 Navigation and Path Following

To assess our skill level and the complexity of the topic that would be assigned during the internship, I was given an exercise on ROS2. This mini-project utilized a ROS2 simulation environment that combined C++, Python, the RViz environment, and TF2. The entire ROS2 project folder was provided by the CRTA laboratory [2]. The mini-project consisted of four steps:

- **Getting started with the simulation and data collection :** The first step was to configure and launch the simulator. I then remotely operated the robot to generate a trajectory, while recording odometry data with `ros2 bag`.
- **Extracting and saving odometry:** I created a ROS2 node (in Python) to subscribe to the odometry topic. When reading the *rosbag*, this node extracts the robot's position (x, y) and yaw angle (yaw), then saves these coordinates to a CSV file.
- **Generating the reference path:** To replay the path, I created a second ROS2 node. This node reads the CSV file and converts the points into a `nav_msgs/Path` message, which is then published to a dedicated topic.
- **Implementation of the *Pure Pursuit* algorithm:** Finally, I implemented a third node that serves as a path-following controller, based on the *Pure Pursuit* algorithm. This node uses the published trajectory and the robot's current odometry to calculate linear and angular velocity commands in real time.

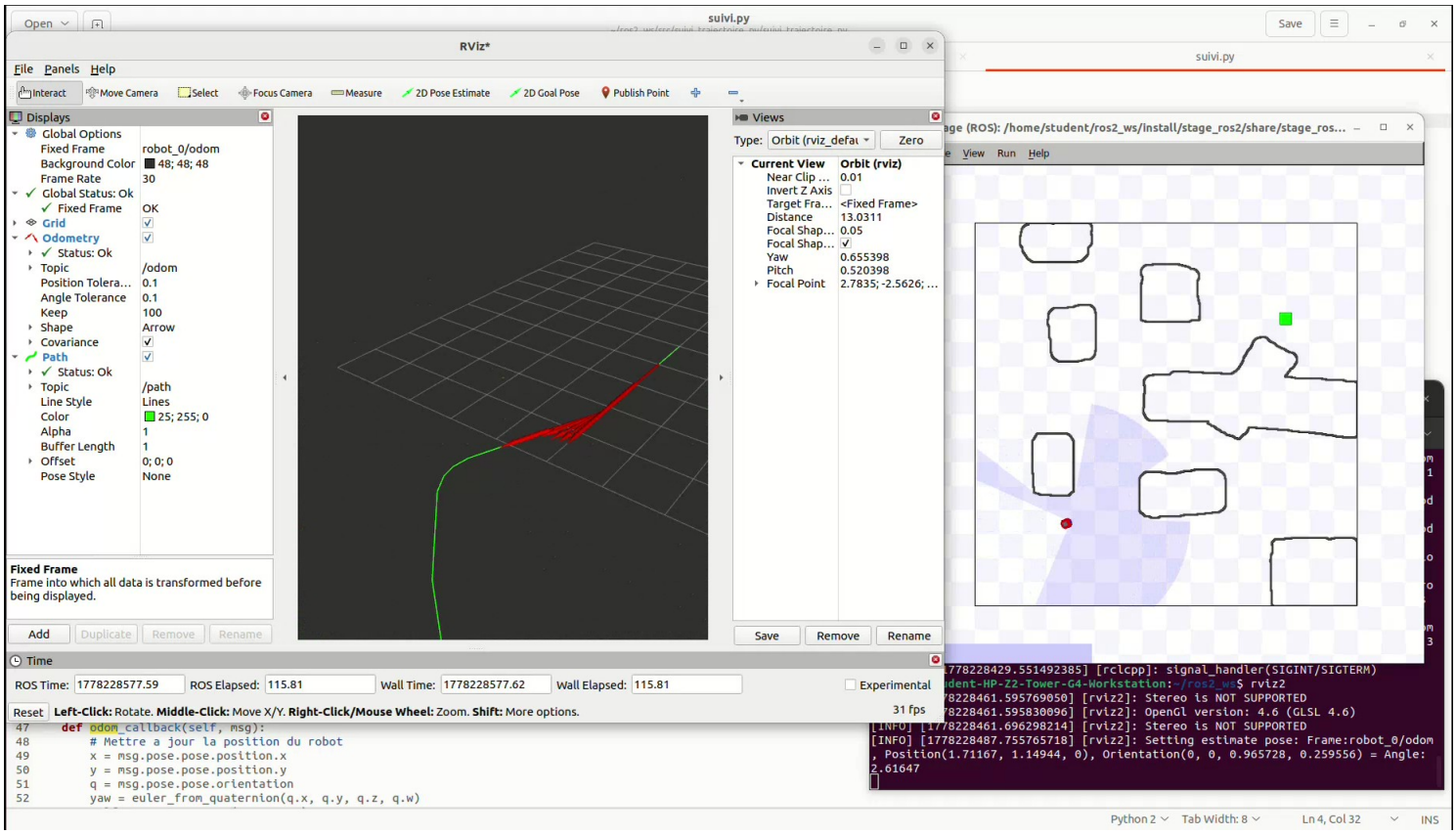


Figure 2.2: RViz visualization of the trajectory reconstruction

In Figure 2.2, the green line corresponds to the recorded path to be followed, and the red vectors are those constructed by the tracking algorithm. In the left window, we see the robot navigating through the space.

This project allowed me to put into practice and validate my mastery of the fundamental concepts of ROS2 (creating nodes, managing *publishers* and *subscribers*, working with *rosbags*), as well as the implementation of control algorithms applied to mobile robotics. I also learned how to use simulation tools such as `RViz`, simulation construction tools such as `tf2`, and how to manage C++ and Python packages.

3 Overview of the Equipment

The initial equipment provided consisted of an inertial measurement unit, two monocular cameras, an optical flow sensor and a mini computer ; I later expanded it by adding a new camera. Not all sensors will be used; the datasheets for those included in the final version of the platform are available in the appendix: [8.5] .

3.1 Operating Mode

Each of the different modules in my equipment operates differently. In this section, we present and discuss these different operating modes.

3.1.1 Computer

I am using an NVIDIA Jetson Orin Nano Super Developer Kit. The following data was taken from the Jetson datasheet available in the appendix: [8.5].

It has an excellent performance-to-power ratio, delivering 67 TOPS with a power envelope ranging from 7W to 25W.

- The 8 GB LPDDR5 memory offers a bandwidth of 102 GB/s, which is a critical factor in avoiding a RAM bottleneck.
- Connectivity to sensors is provided by 4 USB 3.2 Gen2 ports.
- A 4S battery ($14.8V$, $5.500mAh$) powers the Jetson and handles peak hardware power draw ($25W$).

It is an affordable platform that is ideal for prototyping autonomous robots, with native support for hardware acceleration frameworks such as NVIDIA Isaac, but this will be discussed further in the chapter 5.3. Here is a photo of the Jetson 3.1 :

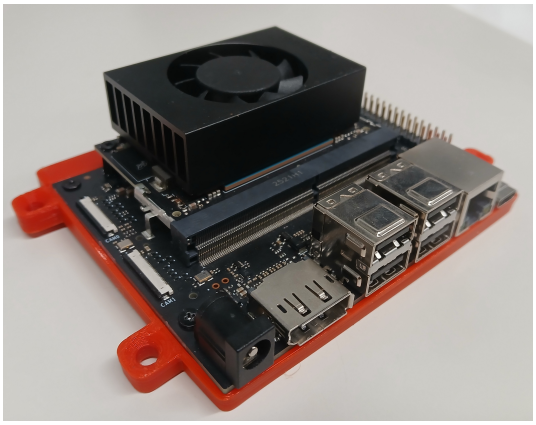


Figure 3.1: Photo of the Jetson

For information on the communication protocol between the computer and the Jetson, please refer to the appendix : 8.5.1 .

3.1.2 IMU

The IMU measures linear acceleration, gravitational force, linear velocity, and orientation using its gyroscope, as well as the magnetic field using its magnetometer. Our *OlixSense X1 Pro* (see picture 3.2) IMU is manufactured by Olive Robotics.

Figure 3.2: Photo of the IMU

It is equipped with an embedded processor and a Linux kernel running ROS2. This means that it natively publishes data topics on its own, and you can subscribe directly to these topics without needing to perform any preliminary processing.

The sensors have a local web page when the sensor is connected to the PC. These can be accessed by typing *http://192.168.8.150/*, where *192.168.8.150* is the sensor's IP address. From this page, I made several adjustments directly in the component's processor:

- I changed the network IP; the problem is that the sensors from Olive Robotics all have the same factory-default IP configuration. Changing the IP will prevent conflicts when multiple sensors are connected.
- I changed the `ROS_DOMAIN_ID` to *0*. This means that all data streams and exchanges will take place on channel number 3.
- I also changed the namespace to *oliveimu* to make it easier to understand.
- The *Frame ID:* also had to be changed to *base_link*. This ID is used to specify the origin of the data stream; *base_link* is the name given to our platform's reference point.

You can find a preview of the sensor's webpage here : [8.5.2] .

The topics, published in ROS2, are ready for use; see 3.3.

```
student@student-HP-Z2-Tower-G4-Workstation:~$ ros2 topic list
/olive/olixSense/x1/oliveimu/acceleration
/olive/olixSense/x1/oliveimu/imu
/olive/olixSense/x1/oliveimu/status
/olive/olixSense/x1/oliveimu/temperature
/olive/olixSense/x1/oliveimu/velocity
```

Figure 3.3: IMU Topics

3.1.3 Cameras

I have a total of 3 different cameras: an *olixVision V1* from Olive Robotics, a *Camera V2.1* from Raspberry Pi, and a *Depth Camera D435i* from RealSense; that is, respectively 3.4 :

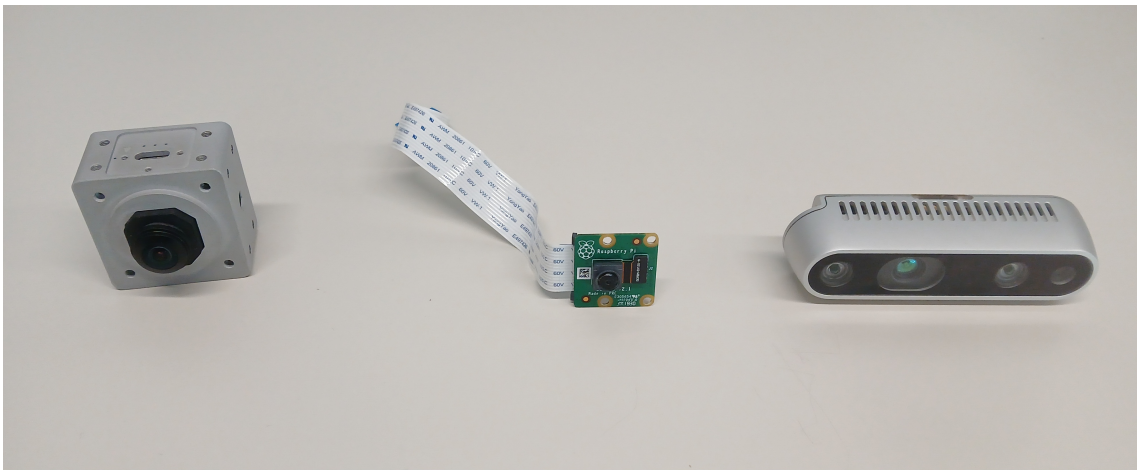


Figure 3.4: The 3 Types of Cameras

The Raspberry Pi camera was automatically ruled out because my Orin Jetson Nano does not have the same connection pins as the Raspberry Pi mini-computers.

The *olixVision V1* and *Depth Camera D435i* cameras have a built-in IMU and processor. They natively publish their topics in ROS2. This makes them perfect candidates for VIO.

For the *olixVision V1* camera, I applied essentially the same changes as in the IMU section [3.1.2], except that the IP address was not changed. This camera has many unique features. Note that the camera's video stream uses a *rolling shutter*, meaning that the pixels are scanned line by line across the image. This caused numerous problems during the camera's intrinsic calibration because it increased the error; see 3.2.

And there is another, more serious problem: its embedded IMU is prone to numerous drifts. After calibrations using *allan_variance_ros*, the returned values appeared consistent, but as soon as the camera was moved and then brought to a stop, its acceleration topic would drift. This was also visible in its pose topic, which is published natively by the processor.

These issues aren't generally specific, but with this one, in just a few movements, we could see a discrepancy of several hundred meters. Unfortunately, I think the camera's IMU is broken. I realized this problem during the VIO tests. So I left all the code related to this camera in my architecture in the hope of being able to replace it. The numerous control and calibration tests with this camera took me several weeks.

I finally decided to use another camera that I hadn't been given initially, the *Depth Camera D435i*. This camera features a color camera—also equipped with a *Rolling Shutter*—as well as two infrared cameras for stereovision and an internal IMU. I decided to use the two infrared cameras because stereovision is an asset for VIO. During the various intrinsic (camera image rectification) and extrinsic (relative position between the cameras and the IMU) calibrations, there were absolutely no problems, and the results were excellent. I therefore decided to integrate the camera into my sensor platform.

3.1.4 Holybro

The *Holybro H-flow* is a module that includes an optical flow sensor, a rangefinder for distance measurement, and an IMU for orientation. The optical flow is used to estimate variations in ground roughness and to calculate a linear velocity value (in 2D relative to the ground).

CAN Bus Hardware Interface

The optical flow sensor transmits its data via the DroneCAN protocol (CAN). It does not have a processor that integrates ROS2 and publishes topics. Interfacing with the PC is done via a *Zubax Babel* USB/CAN adapter.

If you need help with data extraction or getting started with H-Flow, please click here 8.5.3.

H-flow is activated using the following commands :

```
sudo slcand -o -s8 -t hw -S 3000000 /dev/ttyACM0 slcan0
sudo ip link set up slcan0
```

Please note : if the H-flow is unplugged and then plugged back in, its device name will change to */dev/ttyACM1* because the Jetson stores USB device names in memory; you will then need to adjust the command accordingly.

Mathematical Analysis and Binary Structure of the Frame

The optical flow data represents the angular displacement of the image projected onto the PixArt sensor matrix since its last capture. To obtain the sensor's actual linear velocity along the X-axis (V_x), we must subtract the intrinsic rotation measured by the device's gyroscope and project the result using the measured distance from the ground:

$$V_x = \left(\frac{(\text{flow\_integral\_x} - \text{gyroscope\_rate\_x})}{\text{integration\_interval}} \right) \cdot \text{ground\_distance}$$

This principle is documented here [3].

Analysis of the raw binary frame received (`com.hex.equipment.flow.Measurement`), serialized in Little-Endian format, indicates a fixed payload of **21 bytes**:

- **Bytes 0 to 3**: `integration_interval` (type `float32`, sensor integration time).
- **Bytes 4 to 11**: `rate_gyro_integral[2]` (array of 2 `float32`, integrated angular velocity for the X and Y axes).
- **Bytes 12 to 19**: `flow_integral[2]` (array of 2 `float32`, integrated values of the X and Y optical flow vectors used directly for displacement calculation).
- **Byte 20**: `quality` (type `uint8_t`, hardware confidence indicator for surface tracking).

The linear velocities are published on the topic */optical_flow/velocity*. And also, my code already decodes the ID 1050 frame from the rangefinder, converts the data, and publishes it to the topic `/optical_flow/range`. But we will speak more deeply of the sensors data in this section 8.5.3

3.2 Calibration

You can find detailed information on calibrating the various sensors here: 8.5.4.

These calibration steps are essential for controlling our systems. They have allowed us to :

- correct the intrinsic parameters of the images,
- obtain transformation matrices between the cameras and the various IMUs,
- obtain noise and drift measurements that will be used in our model's covariances to measure our confidence levels in certain parameters or sensors,
- generate *.yaml* files that consolidate all this information in a format optimized for VIO.

3.3 Limitations

The extrinsic calibrations between the cameras and the *OlixSense* IMU were not optimal. This error is due to the camera-IMU leverage arm and, to a small extent, to the time lag caused by image decompression. The resulting transition matrices are poor. This means that the IMU cannot be used with the VIO and will be incorporated into the overall system via UKF or EKF sensor fusion.

It's a shame because the IMU can publish data at a high frequency (I've limited the IMU topic's publication rate to $500Hz$), and that's a major advantage for the VIO. Since it will be integrated into the UKF/EKF, an IMU with such a high update rate isn't necessary, so it can be replaced with a lower-cost IMU. Additionally, for your information, camera-based IMUs update at $200Hz$, which is sufficient for VIOs.

We will likely have to settle for using the *D435i* rather than the *OlixVision*, since the latter's onboard IMU is prone to significant drift, even after calibration.

As for the *Holybro H-flow*, its range data is excellent, but the calculated velocities V_x and V_y are of reduced quality due to gyroscopic noise.

The next step is now to develop several platforms to integrate these sensors.

4 CAD Modeling of the Sensor Platform

This chapter details the creation of sensor platforms produced via 3D printing using 75mm/1000g PETG. The CAD model ensures that the spatial relationships between the IMUs, the camera, and the optical flow sensor are rigidly defined in order to preserve the integrity of the extrinsic calibration matrices. The plan was to fabricate the platforms from metal using CNC machining, but due to time constraints, this was not done. We will see that, in our situation, there are three possible types of platforms.

4.1 Flexible Experimentation Platform

The experimental platform is not intended to be mounted on a drone. I used a 48 cm aluminum profile to which I attached 3D-printed modules housing my sensors. This modular structure is flexible, allowing for rapid reconfiguration and optimal positioning of the NVIDIA Jetson Orin Nano and the sensor suite, thereby ensuring unobstructed fields of view and precise tracking of OptiTrack markers without structural obstruction. Furthermore, if future tests require the integration of a second camera, a LiDAR, or even a GPS, this can be done easily. Here is an overview of the 4.1 platform:

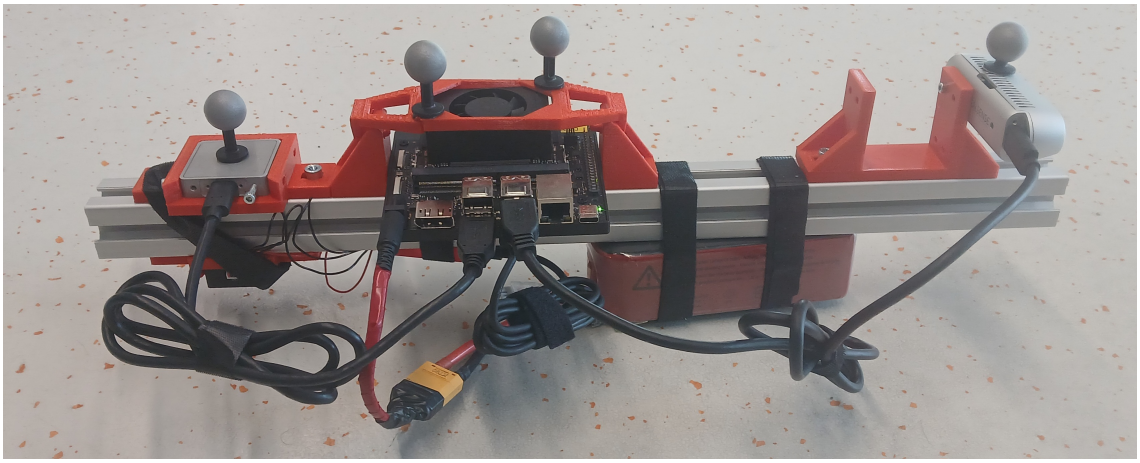


Figure 4.1: Flexible Experimentation Platform

At the top left is the *OlixSense* IMU, and at the top right is the *D435i* camera (with a slot for the *olixVision*, see here), in the top center is the Jetson, below that is the battery, and at the bottom left is the *Holybro*. There are also 5 OptiTrack markers (4 on top and 1 in the background).

Constraints

The main challenge is heat dissipation, but since the device was only used for up to 30 minutes at a time, this isn't really a problem.

- Heat Dissipation : Fusion sensors generate significant heat during high-performance computing and AI operations. The *Olixvision* camera and the *OlixSense* IMU generate so much heat that it is impossible to hold them in your hands. In particular, they must be mounted directly onto a metal chassis or

mounting plate to ensure efficient heat transfer and stable operation, as prolonged exposure to excessive heat can impair performance or cause irreversible damage. This is why the platform, in its final form, must be manufactured using CNC machining.

- Spatial Alignment: All sensors must be very well fixed with high tolerance to prevent micro-movements that would corrupt the TF2 tree and induce drift in the Extended Kalman Filter (EKF).

Models

You can find the CAD drawings on GitHub by clicking here, and the component datasheets here. The CAD assembly integrates the following components :

- Camera : The camera has dimensions of $38.0mm \times 38.0mm \times 38.3mm$ and a weight of $65g$.
- IMU / AHRS : It measures $40.0mm \times 40.0mm \times 10.0mm$ and weighs $36g$.
- Optical Flow: It has a weight of $15.2g$ and must be mounted facing the ground to measure surface displacement.
- Compute Unit: The NVIDIA Jetson Orin Nano is modeled with dimensions of $103mm \times 90.5mm \times 34.77mm$, acting as the main processing hub at the center of gravity.

Code Compliance

To share the sensor positions on the platform with our ROS2 architecture. We use an XML Macro (XACRO) file (*sensor_platform.urdf.xacro* where URDF means Unified Robot Description Format) to define the "robot's" geometry. This must comply with the REP-105 standard (ROS Enhancement Proposal 105), which defines the naming conventions and semantics of coordinate reference frames.

For the modular platform, its URDF is stored in *xacro_for_modular_platform*. Our ROS architecture only calls the file *sensor_platform.urdf.xacro*, so if you change platforms, you will need to copy and paste the text from one file to the other.

We must then define the "base_link" node, which is the fixed reference frame attached to the robot's main body. The global reference frames "odom" and "map" are properly externalized so that they can be managed by the "robot_localization EKF/UKF" and "SLAM/VIO" nodes.

Transformations (TF2) must follow ROS's right-hand rule. For example, the IMU (*imu_link*) is defined with the Z-axis pointing upward using the rotation $rpy = "0\ 0\ \pi/2"$, and the optical flow sensor (*optical_flow_link*) is tilted downward using $rpy = "\pi\ 0\ -\pi/2"$ so that it faces the ground.

Notes: The onboard IMU of the *OlixVision* did not follow this right-hand rule—the gyro appeared to be inverted—and it was impossible to correct this from the local web server.

The sensors are positioned using translations and rotations relative to the *base_link* coordinate system; this is convenient because the extrinsic calibrations return transformation matrices (rotation + translation). You can view the URDF by clicking here.

4.2 Drone Platform

The goal of the system is to be mounted on a drone; therefore, the model must be ultra-compact, easy to disassemble, and easy to attach. The designed model is shown in 4.2. Weighing 1097g (see here); including sensors, battery, Jetson, screws/nuts, cables, and OptiTrack markers; it can be mounted on a drone capable of carrying a payload of 1.2 kg (add the weight of the mounting hardware to the drone's payload capacity).

Figure 4.2: Embedded Platform for drones

The platform has four mounting holes along the edges so it can be mounted on drones.

Constraints

Space and weight are the main constraints. I drew inspiration from the way sensors are mounted on the modular platform. Each component has its own CAD mounting model; they are attached to the platform via an adapter, and the total thickness does not exceed $1cm$.

I therefore designed a $19cm \times 8cm$ plate intended to accommodate each of the sensors. Spaced from the platform by 3-mm nuts, the components can be stacked on top of and beneath one another without any mounting issues. This spacing is also an advantage for the IMU (and perhaps the *OlixVision* if it is ever reintegrated) because it tends to get extremely hot, and this spacing will allow heat to dissipate.

As for the Jetson, as you can see in 4.1, it does not have any mounting points on its base mount and must be "secured." To fix this, I replaced the Jetson's mount. I separated the mini-computer from its board and its FPC Wi-Fi/Bluetooth antennas. I obtained the CAD models of the structure, which I then modified to incorporate four mounting points.

Models

The CAD models are available on GitHub by clicking here. You can also find here the mounting brackets for Spot, the robotic dog from *Boston Dynamics* [13].

Code Compliance

The URDF file for this platform is located in *xacro_for_drone* (see here). Like the file for the modular platform, it serves as a text file repository and is not called by the ROS2 architecture. Therefore, when using the platform, you must copy and paste the contents of *xacro_for_drone* into *sensor_platform.urdf.xacro*.

Additionally, note that the code must be adjusted depending on the drone being used. For example, the code can remain unchanged if the platform is integrated as-is onto a flying drone, but for *Spot* [13], the Holybro must be placed below or behind (as shown in 4.3) the dog in order to report the linear velocities in the x and y directions. Furthermore, for *Spot*, the altitude parameter can/must be commented out in the merge file to prevent errors from being introduced if the dog's altitude varies (such as when the dog climbs stairs or obstacles).

Figure 4.3: Embedded Platform on Spot with Holybro fixed at the back

5 Workspace Architecture

The *sensor_platform_ws* workspace contains various packages. Here is a brief explanation of their roles below:

- `hardware/`: This folder contains hardware drivers in the form of submodules or external packages (`realsense-ros`) as well as the CAN bridging package (`uavcan_bridge`).
- `olive_networking/`: Package dedicated to the low-level configuration of the native sensor network (IP initialization scripts, DDS configurations for real-time, and `udev` rules for naming IDs).
- `platform_description/`: Contains the geometric descriptions of the platform via URDF and XACRO files, structured in strict accordance with the "REP-105 convention".
- `platform_bringup/`: Contains the launch scripts of the system (`.launch.py`) as well as all parameter configuration files (`.yaml`) needed for enable system control .
- `platform_processing/`: Package for data formatting, stream synchronization, and high-performance computing.

In this chapter, we will focus on the entire pipeline, from its initialization to the merging of sensor topics. When discussing the pipeline, we'll generally refer to *system.launch.py*, which is the Python launch script for the entire system. You can view it here.

5.1 Overall Architecture Adjustments

There are always minor adjustments to be made; it's not enough to simply connect the sensors to the Jetson and start the system. In this section [8.6] , we'll discuss fine-tuning the architecture to avoid various communication issues or problems with topic handling.

5.2 Data Acquisition Chain

In this chapter, we will focus on how to retrieve and republish data from topics. We must ensure that the topics used by our system have the correct *frame_id*, that certain topics are synchronized, and that the communication protocols between *subscriber* and *publisher* are followed.

5.2.1 Sensor Holybro

This subsection discusses data acquisition as implemented in the *dronecan_bridge_component.cpp* file, available at here. Here, the data streams *vx, vy, and range* are retrieved, which have been decoded using libcanard as explained in section 3.1.4. They have been pre-filtered by a low-pass filter to reduce noise. We construct two separate topics for altitude and linear velocity.

For velocities, we overwrite the sensor's *timestamp*—which corresponds to the manufacturing date—and replace it with the Jetson's timestamp. We associate the topic with the *frame_id "optical_flow_link"* mentioned in the URDF, along with the x- and y-velocity values and the covariances. The message type for the topic is *TwistWithCovarianceStamped*; it is essential to include the covariances. For unpublished values, these are set to 10000.0 so that no fusion algorithm will link them. The covariances of the linear velocities are dynamic; I use the formula :

$$dynamic\_cov = base\_cov * quality\_factor * dist\_factor$$

Where $base\_cov$ is a covariance estimated at 0.05, $quality\_factor$ is the optical flow quality ratio of the order of 10^{-1} ($quality\_factor = \frac{Max\_quality=255}{quality\_published}$, and $dist\_factor$ is either the published distance squared or 1 (to avoid getting 0 when the drone is on the ground). This results in a covariance that oscillates between 3 and 0.1 during testing.

For distance, we also overwrite its *timestamp* and then pass its *frame_id* to *odom*. An error occurs in the UKF if we link the topic to the *base_link* because the message is of type *pose* and the UKF requires that it be published in *odom*. We must then perform the URDF translation in the file. We then use the *OlixSense* IMU to retrieve the exact roll and pitch angles of the platform. We then use the formula :

$$z_sensor = distance * cosf(r) * cosf(p);$$

to obtain the hypotenuse of the 3D triangle and thus the sensor height, independent of its angular variation.

We perform a qualitative analysis of the height by comparing the range topic to Opti Track. By varying the orientation, we observe the variation in 5.1:

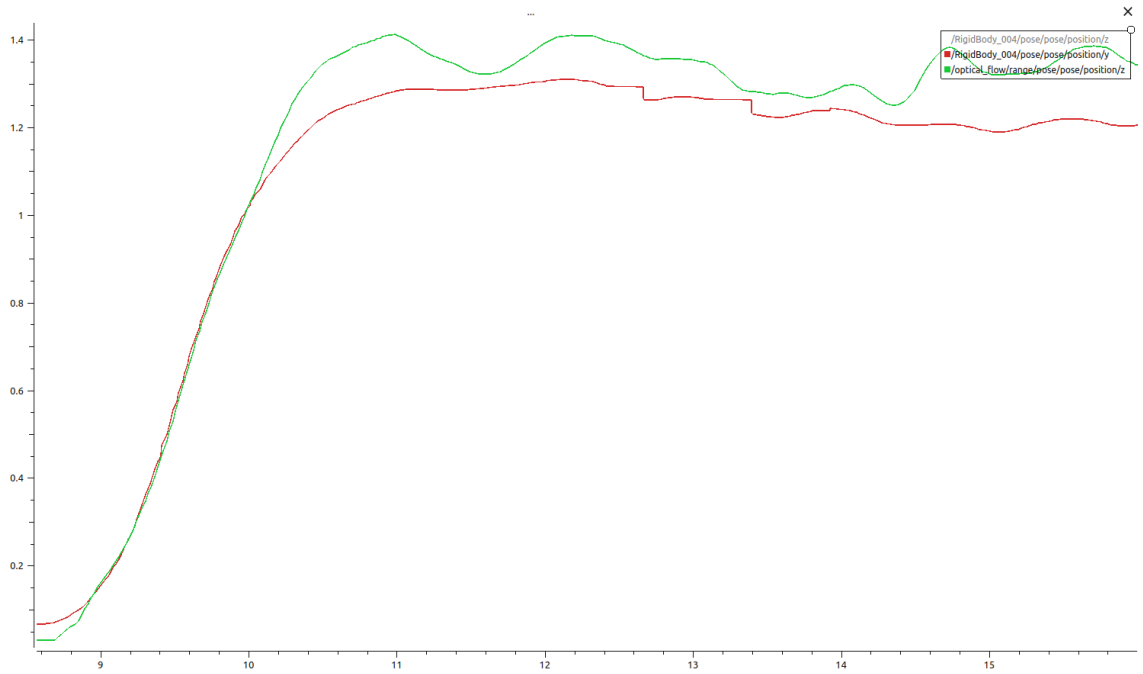


Figure 5.1: Value variation Holybro and OptiTrack

The OptiTrack altitude curve is shown in red and the Holybro curve in green. The offset is $5cm$ at startup; as altitude increases, the two curves overlap briefly, and then the offset reappears. We can see several fluctuations in the green curve, while the red curve remains steady because, in this test, the height does not

change and only the roll and pitch angles vary. Upon measurement, I note that the maximum error is $13cm$. Subtracting the $5cm$ offset at startup leaves only $8cm$, which is acceptable given that the angles are ~ 30 degrees.

The covariance for z was set to 0.001 because the results are very close to reality.

As for the QoS (or Reliability) policy, both topics are published as Reliable; the EKF/UKF can subscribe to Best Effort without having to send an acknowledgment of message receipt. These two topics are:

- */optical_flow/range* publishes at $50Hz$ to *odom*,
- */optical_flow/velocity* publishes at $65Hz$ to *optical_flow_link*.

5.2.2 Sensor Camera

We have two cameras; since the *D435i* has shown more promising results than the *OlixVision*, we will focus on the former in this section. You can find explanations about the *OlixVision* data acquisition chain here: 8.7.1.

The *RealSense D435i* topics are not used by the EKF/UKF but only by the VIO, which publishes the odometry topic. Therefore, there is no need to retrieve the topic to change its *timestamp*. The camera data is configured directly from *system.launch.py*:

```
1  'camera_name': 'd435',
2  'enable_gyro': True,
3  'enable_accel': True,
4  'unite_imu_method': 2,
5  'gyro_fps': 200,
6  'accel_fps': 250,
7  'depth_module.emitter_enabled': 0,
8  'enable_depth': False,
9  'enable_color': False,
10 'enable_infra1': True,
11 'enable_infra2': True,
12 'depth_module.profile': '848x480x30',
13 'enable_sync': True,
14 'initial_reset': True,
15 'serial_no': '108322073329',
16 'publish_tf': False,
17 'global_time_enabled': False
```

Lines 1–6: Since the D435i has a gyroscope and an accelerometer that sample at different frequencies ($200Hz$ and $250Hz$). Method 2 (*unite_imu_method*) instructs the driver to perform linear interpolation to merge these two data streams into a single topic of type *sensor_msgs/Imu*. Without this, OpenVins or robot_localization would reject the data.

Lines 7–9: We disable the infrared emitter (*emitter_enabled*: 0, which adds a point cloud that distorts feature tracking) and disable depth calculation; the goal is to conserve USB bandwidth and ARM CPU resources.

Lines 10–13: Only the two infrared cameras (left and right) are enabled at a resolution of $848 \times 480px$ and a frequency of $0Hz$; note that these do not use a *rolling shutter*. The *enable_sync* parameter forces the

RealSense ASIC chip to hardware-lock the timestamps of the two infrared images to the IMU to ensure that the pixels correspond perfectly to the inertial measurements to the millisecond.

Line 16: If the parameter were set to True, the RealSense would start publishing its own transformations between its lenses. However, in our architecture, it is the *sensor_platform.urdf.xacro* file that publishes the TF tree. We want to avoid concurrent transformations.

Lines 14, 15, 17: We flush the on-board RAM of the camera specified by its serial number at startup to prevent frame drops during rapid restarts. We set *global_time_enabled* to False so that the driver generates a timestamp based on the Jetson's system clock (this->now()) at the millisecond when the USB packet is received by the CPU.

When combined with the calibration files (camera and IMU, see 8.5.4), the camera topics will be ready for use.

5.2.3 Sensor IMU

The acquisition and preparation for use of the IMU topics are handled in the file *imu_sync_component.cpp*. We simply retrieve the topic */olive/olixSense/x1/oliveimu/imu* published at $1000Hz$ and then apply the following modifications:

- Changing its *frame_id* to *imu_link*—this can also be done from the sensor's local web page,
- Overwriting its *timestamp* with the Jetson's timestamp
- Combining the variance values from the accelerometer and gyroscope with the values obtained during calibration using *allan_variance_ros* (see 8.5.4) for angular velocities and linear accelerations; and the values listed in the datasheet for angular orientation,
- checking for a zero quaternion (where w, x, y, z are at 0.0, which is the case when the sensor starts up if it is stationary). If this is the case, we force a neutral orientation ($w = 1.0$) and set *orientation_covariance[0] = -1.0*. In ROS2, this explicitly instructs *robot_localization* (EKF/UKF) to ignore the orientation data, thereby preventing the matrix calculation from crashing.
- Reposting the topic under the name */imu/fused* (at the beginning of the internship, the plan was to fuse multiple IMUs on the platform, so there were supposed to be *fused1, fused2, ...*; that is why the topic is named this way) at $500Hz$.

The IMU topic is now ready to be integrated into a sensor fusion filter for *robot_localization*.

5.3 VIO Odometry

In this section, we discuss how to construct an odometry track using VIO. VIO, or Visual-Inertial Odometry, is a method that combines a camera and an IMU to estimate a 6D pose (xyz and rpy).

To do this, I decided to use the *OpenVins* package ([8]), which leverages the Jetson's CPU power. There are other types of VIO, such as *isaac_ros_visual_slam* ([9]), which is adapted for the Jetson and developed by NVIDIA; this one uses the mini-computer's GPU acceleration and consumes less CPU power. However, after successfully completing several tutorials on different datasets, I found that working with *IsaacRos* in a Docker environment takes time to get the hang of and is complicated. Due to time constraints, I decided not to delve deeper into this VIO and instead focus on OpenVins. Furthermore, *IsaacRos* requires stereovision and does not work with a monocular camera such as the *OlixVision*.

All the files related to the OpenVins configuration for our ROS2 architecture are available in this folder here. You can also view the original configuration files in the OpenVins source code by clicking here.

5.3.1 Control of input and output flows

Notes: The test results with the *Olixsense* were extremely poor; the pose diverged almost instantly. We are definitively ruling out the use of this camera and, above all, its onboard IMU.

For the *D435i*, we decided to work with infrared image stereovision. To do this, I followed the tutorial [4] using another camera from the *RealSense* line, the *t265*.

The input streams are configured in the following configuration files: *kalibr_imucam_chain.yaml* and *kalibr_imu_chain.yaml*. This is where you enter the results of the calibration performed with kalibr (see 8.5.4). These files contain the transition matrices *cam0-cam1*, *cam0-imu*, *cam1-imu*, and other data. For the IMU configuration file, be sure to multiply the *white noise* by 5 and the *random walk* by 10, as discussed in 8.5.4.

Now that the input streams are calibrated, we're going to link the output stream to our *base_link* published by our URDF. Rather than subscribing to the output odometry topic and republishing it by changing its *header.frame_id*, I chose to modify the OpenVins source code.

In the *open_vins* package, we modify lines 318 and 353 of the file *open_vins/ov_msckf/src/ros/ROS2Visualizer.c*
The new *header.frame_id* is *d435_gyro_optical_frame*, which refers to the D435i's IMU in our *base_link* from which the odometry was published.

We determined the name of the *frame_id* from the following file 8.8.3.

5.3.2 OpenVins Calibration

VIO calibration is performed using the *estimator_config.yaml* file.

Here are the modified settings:

- *calib_imu_g_sensitivity*: The IMU is subject to micro-deformations and thermal variations that affect how the gyroscope senses gravity. Enabling this parameter allows OpenVins's MSCKF filter to dynamically estimate and correct this error during flight. This reduces altitude drift over the long term.
- *try_zupt* and *zupt_max_disparity*: Enabling ZUPT sets the velocity to zero if the algorithm detects that the platform is no longer moving. Setting the value to 0.4 makes the visual detector more stringent: the system will require clearer visual stillness before triggering this lock, preventing false movements caused by noise from the *CMOS* sensor.
- *init_dyn_use*: Enabling dynamic initialization allows the filter to converge and start VIO even if the drone is already in motion. By default, OpenVINS requires the platform to remain completely stationary for the first few seconds to calculate the IMU's static biases. On a drone, this is often impossible.
- *use_mask*: The original file was configured for Intel RealSense T265 fisheye cameras. These masks were used to ignore the black edges of the image. Since we are using the D435i, these masks are unnecessary because the images are already rectified.

5.3.3 Quality Assessment

You can watch here a sample video illustrating the output generated by OpenVins when we enable all these settings. We run a test, during which we record the data in a rosbag using *record_dataset.launch.py* (see 8.11) before pushing it to our PC. We then compared the results obtained with OptiTrack (see tutorial 8.9).

Once on the computer, the rosbags are placed in a folder where the *evo* package [5] has been installed; we then compare the two pose topics using the *evo* package (see 8.10). We execute the error measurement command after aligning the two trajectories 5.2.

First, in 5.2a, we see that the average position error (APE) is $\sim 7.4cm$, which is excellent given that the distance between the camera and the origin of the *RigidBody* is approximately $10cm$. The RMSE value is $9.6cm$. This metric penalizes large errors quadratically. It is this value that we should aim to minimize by adjusting the configuration parameters. However, we note that the **median** error is approximately $3 \sim$ cm. The fact that the median error is significantly lower than the RMSE indicates that during the vast majority of the acquisition time (50% of the time), the error was small.

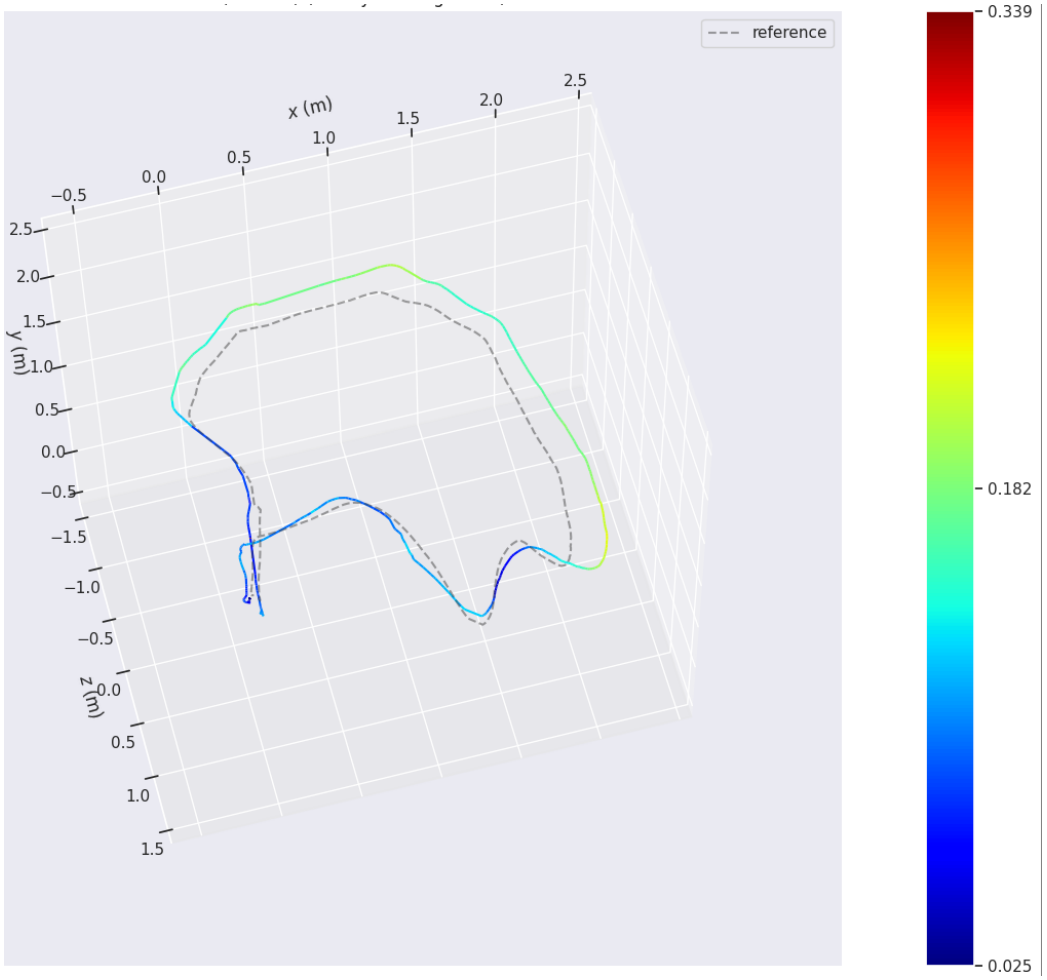
To explain this, simply look at Figure 5.2b; we see that the error (gray curve) is nearly zero during half of the recording time. Comparing this figure to Figure 5.2c, we see that this corresponds to the platform's rotation period.

```
APE w.r.t. translation part (m)
(with SE(3) Umeyama alignment)

       max      0.339130
      mean      0.073861
    median      0.027706
       min      0.024860
      rmse      0.096619
       sse      14.142731
       std      0.062287
```

(a) Translation Error Statistics

(b) Changes in the APE Over Time



(c) 3D Trajectory and Error Mapping

Figure 5.2: Overall Assessment of the Absolute Pose Error (APE) for the Translation Component Using Umeyama Alignment

Then, using the overlay command without error analysis (see tutorial 8.9), we obtain the following graph, with the OpenVins pose shown in blue and the OptiTrack pose in gray:

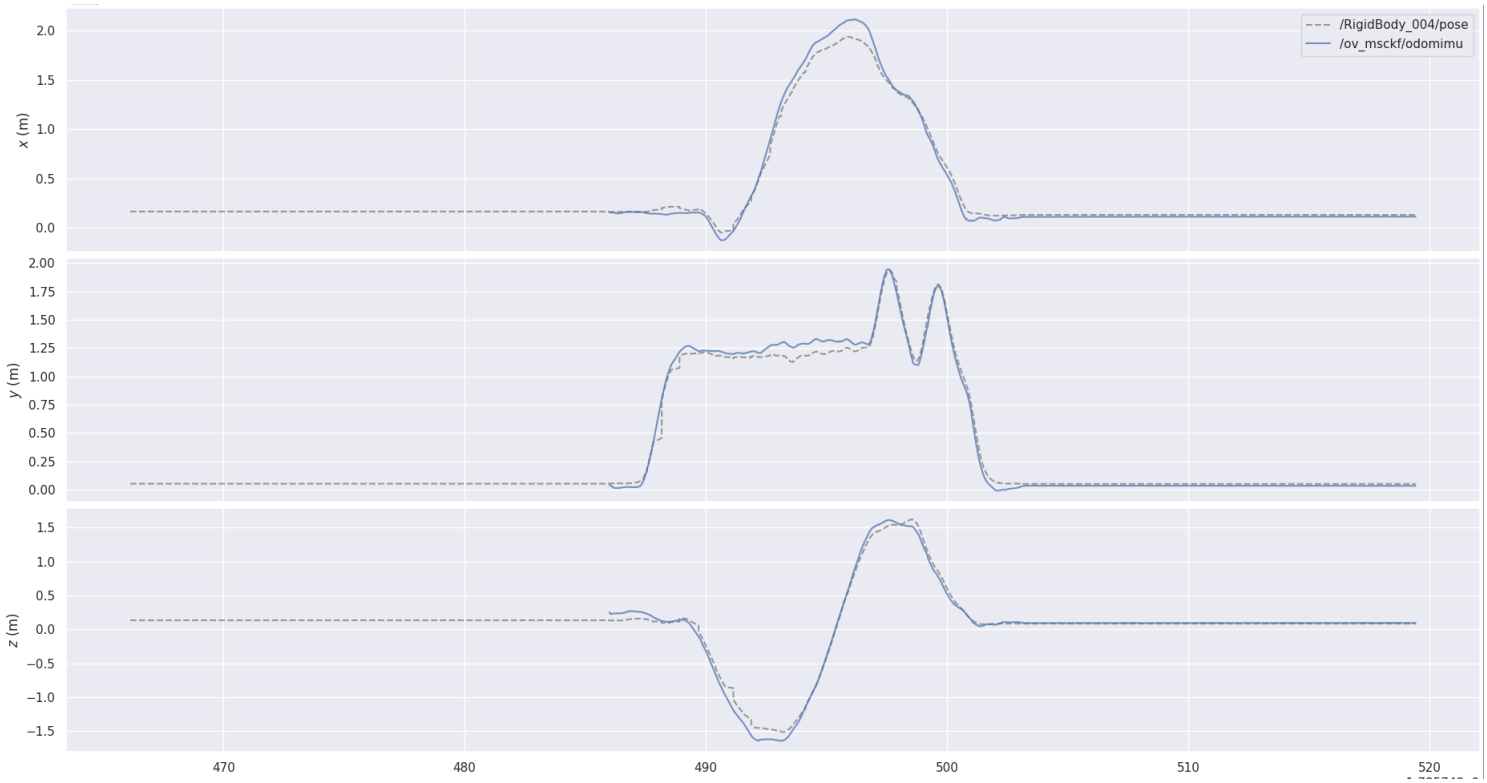


Figure 5.3: Analysis of Position Error in the x, y, and z Directions

A y-offset (altitude) occurs when the camera's altitude remains constant. Offsets occur in the x and z directions at the peaks of curves; comparing this to 5.2c, we can see that OpenVins overestimates its pose on the outer part of its trajectory.

Explanations:
The worst enemy of a VIO algorithm is coming to a standstill. If the drone is resting on the ground or moving only slightly along one of these axes, the natural noise from the IMU will be mathematically integrated and cause the odometry to "drift."

5.4 Sensor Fusion

Our goal in this section is to compensate for the slight position drift of the OpenVins pose topic: To do this, we will fuse our sensors into a filter. There are different types of filters; we will focus on those from *robot_localization* [10]. This package estimates poses based on the nonlinear states of the system; it relies on *Kalman filters*. Here are three kind of Kalman filter:

- **EKF**: The Extended Kalman Filter is the most widely used algorithm in industry for nonlinear systems. It linearizes the kinematic model around the current estimate by calculating partial derivatives (Jacobian matrices). The EKF uses a predictive model and sensor covariance matrices to fuse imperfect data (OlixSense and H-Flow IMUs) and estimate the most probable position, velocity, and orientation (odometry) at time t.
- **UKF**: The Unscented Kalman Filter is a nonlinear alternative that completely avoids the computationally intensive and sometimes unstable calculation of Jacobians. Instead of linearizing the model, it selects a set of deterministic points (sigma points) around the state mean and propagates them through the actual nonlinear functions. It is mathematically more accurate than the EKF when dealing with strong nonlinearities (such as sharp turns, which is an advantage for our drone), hence its inclusion in *ukf_fusion.launch.py* to compare its flight performance against the EKF.
- **InEKF**: The Invariance-Extended Kalman Filter is a geometric extension of the EKF based on Lie group theory (such as $SE(3)$). Unlike the classical EKF, where the estimation error depends on the robot's trajectory, the InEKF formulates an "invariant" error. This ensures that the filter matrices remain constant and do not diverge, even during high accelerations. In our specifications, it is designed to optimize the fusion between the IMU and the optical flow velocities during aggressive maneuvers.

Since the UKF is more robust than the EKF, we will focus more on it. As for the InEKF, there is no *.yaml* configuration file; it is run via *Python, MATLAB, or C++* scripts. In the publication [11] (see introduction, page 3), it is stated that their state-updating UKF is more robust than the InEKF.

In this other publication [12], the author discusses the superiority of the InEKF over the EKF because, being less sensitive to nonlinearities, it converges to the true ground position while the EKF drifts. If the UKF is prone to this type of drift, then implementing an InEKF could correct it. However, we are not sure whether to implement it because writing this kind of script and the accompanying math is extremely time-consuming.

The *.yaml* configuration file for the EKF/UKF filters is a key component of our architecture. Unfortunately, there are no tutorials on how to adjust or configure them. This was an empirical process I went through myself. Through more than 60 tests (for the UKF alone), I learned to fine-tune the various parameters in the file one by one.

5.4.1 UKF

You can find the UKF configuration file here.

Here are the key points of the file

- *odom0*: We retrieve the x, y, z and r, p, y positions published by OpenVins, topic */ov_msckf/odomimu* ($200Hz$). A property of the UKF is that the values in the matrix must be independent of one another (particularly the covariances associated with the measurements) in order to prevent crashes. Therefore, we do not retrieve the velocities and accelerations for *odom0*.
- *odom0_pose_rejection_threshold*: The threshold between the UKF estimate and OpenVins must be very large to prevent crashes in cases where the camera is temporarily blind, as the algorithm may diverge and then converge again.
- *odom0_relative*: This does not determine the initial pose; the goal is to avoid initializing the altitude to 0.
- *odom1*: We do not use this input, but we should integrate a GPS here.
- *pose0*: We only implement the z position measured by the Holybro, topic */optical_flow/range* ($50Hz$).
- *twist0*: We add the linear velocities in x and y measured by the Holybro, topic */optical_flow/velocity* ($65Hz$).
- *imu0*: We fuse $r, p, V_r, V_p, A_x, A_y, A_z$ from the *imu/fused* ($500Hz$) topic. The IMU's linear velocities are estimated from the accelerations, so they are not needed. However, we must publish the roll and pitch angles to provide the initial orientation. We let OpenVins publish the yaw angles.
- *imu0_relative*: designates the IMU's initial orientation as that of the system.
- *_queue_size*: Since our data is published at high frequencies, we increase the subscription queue size to increase the number of fused data points.
- *imu0_linear_acceleration_rejection_threshold: 0.4*: We limit the values published by the IMU to low accelerations (for slow turns) because these values create pose noise.
- *use_control: false*: We do not send control commands to the system.
- *process_noise_covariance*: This establishes the level of confidence we have in the system's values. I chose these values by analyzing the graphs of published values against ground truth; I identified periods of drift, then analyzed the associated covariance values for those periods, and reduced my covariance tolerance so that during those periods the algorithm does not rely on the sensor but calculates the pose values instead. Note: You must try not to go below the published covariances during periods without drifts; otherwise, you will lose the quality of the pose estimate.
- *initial_estimate_covariance*: The matrix is not a 1×15 but a 15×15 (otherwise the UKF crashes). The values in this matrix correspond to the covariance estimates before the first topic is received. By setting the values to 1.0, we tell the algorithm to wait before estimating the position. The acceleration values are set to 0.5 so that when the accelerometer gives a sudden spike at startup and the position drifts (the VIO needs this spike to activate), the accelerometer's return to normal causes the position to converge again.

5.4.2 EKF

You can find the EKF configuration file here. It has exactly the same constraints as the UKF. The changes made are therefore the same as those for the UKF.

When creating the EKF, I noticed that the parameter *imu0_remove_gravitational_acceleration* was set to *false* in the UKF. It is essential to set this parameter to *true* because the *OlixSense* reports a gravitational acceleration of 9.81. The fact that this parameter was disabled in the UKF was the cause of numerous position divergence errors, particularly when the platform remained stationary for too long.

Since the EKF was integrated late in the project and few tests were performed with it, we will focus more on the UKF.

6 Experimentations and Analysis

In this chapter, we discuss in detail the various tests that were conducted. During the analyses, we use the OptiTrack coordinate system, where Y is the altitude and X and Z are the ground axes. The test area is of $8 \times 5m$. Speed-related data will not be analyzed because the data published by OptiTrack is inaccurate. The same applies to orientation : since the difference between the rigidBody and the base_link is too large, the value graphs are unusable. However, the orientations analyzed during testing appeared to correspond exactly to reality; you can find the tutorial for evaluating orientation in the appendix (see 8.10). The first four sections cover the modular platform, and the last section covers the marked platform. Since the EKF filter was implemented at the end of the internship, we have limited data; therefore, we will analyze only the EKF pose topic in the first part. You can find all the recorded rosbags on the GitHub page by clicking here. In each of the rosbags, you'll find screenshots and videos showing the error analysis performed by *evo* [5].

6.1 Gradual Sensor Fusion

In this section, we compare the sensor fusion of the *EKF/UKF*—first *VIO + Holybro*, then *VIO + OlixSense*, and finally *VIO + Holybro + OlixSense*—to the ground truth from OptiTrack.

6.1.1 VIO + Holybro

The *UKF30* rosbag is available here. The */optical_flow/velocity* and */optical_flow/range* topics published by the DroneCAN bridge are used to correct the estimates of horizontal velocity and altitude.

From the error analysis (see here), we can see that the RMSE error of the EKF/UKF is $0.5cm$ greater than that of the VIO. This does not mean that the Holybro implementation is poor, as the variation in error is too small. We also note from the pose analysis of the axes (00:00:08 in the video, see here) that a constant pose offset appears at the beginning of the test and that it is corrected over time.

6.1.2 VIO + OlixSense

The *UKF31* rosbag is available here. The topic used by the sensor fusion files is */imu/fused*. As discussed in 5.4, it must implement/correct the roll, pitch, roll rate, pitch rate, and linear acceleration values.

The 2D position values (see 00:00:10 in the video) are much better when the platform is far from its starting position. However, the closer it gets to the starting point, the larger the error becomes. On the X axis, for example, the same offset is observed at the beginning as at the end of the test.

We can see that the altitude (Y) positioning correction is poor without the Holybro. However, based on the *RMSE* error analysis (see photo), the EKF/UKF are $2cm$ more accurate than the VIO.

6.1.3 VIO + Holybro + OlixSense

The *UKF29* rosbag is available here. A divergence in the UKF's position occurs at launch. This is ignored because the algorithm reconverges instantly afterward. Aside from that, both filters are usable. We note that their pose estimates are very close.

In the RMSE error analysis, we see that the EKF once again has an error $2cm$ smaller than the VIO, while the UKF has an inflated error due to its initial drift. I chose not to crop the rosbag to hide the error in order to show that sometimes the filters diverge during VIO startup. And the EKF has $10cm$ of error with the OptiTrack setup, which is negligible when you consider the difference in origin between the RigidBody and the base_link defined in the URDF.

For the attitude analysis, we see that on the planar axis the offsets are minimal, and that for altitude, the filters oscillate between the heights provided by the VIO and the Holybro. These oscillations are beneficial and, in many situations, correct for position drifts.

6.1.4 Summary

The combination of *OlixSense* and *Holybro* is beneficial for pose estimation. The *UKF and EKF* algorithms from *robot_localization* are well-suited to various situations. Across all tests, we observe that the position reported by the UKF is always very similar to that of the EKF. The position estimated by the algorithms has an error of $10cm$ in cases where the UKF/EKF do not diverge (and then converge again) at startup (see here). In the following sections, we will focus solely on the UKF, except for the tests with *Spot*.

6.2 Initial Dynamic Test

In the *UKF2* rosbag (see here), we perform a basic test, and a significant variation in the initial pose is observed. The goal is to validate the convergence of the UKF covariance matrix at startup. The VIO requires camera motion to report the position. The IMU's acceleration causes the position to diverge due to the sudden movement, since it is the only data being reported, but the pose is then corrected thanks to the convergence of the covariance matrix, to which the OpenVins values have been added.

6.3 Complex Trajectories

We test our algorithms on 2- and then 3-minute test runs.
The rosbags *UKF16_c* (see here). To fill the space, we recorded a grid-pattern trajectory for 180 seconds. At first, the turns are too tight; the VIO loses its position and does not turn during the first three turns. Since the turn is not wide enough, the VIO thinks it is a static turn, and the first three round trips overlap. However, a wider turn follows, and the drift problem is then resolved.

In the position analysis video (see here), at 00:00:14 we see the period during which the positional drift in the X and Z axes was significant. During the remainder of the period, even though the algorithm converges again, a shift of about fifteen centimeters is observed along the X axis.

In the error analysis video (see here), the red and yellow poses at the beginning—where the error was greatest—are clearly visible. Observing the error along the rest of the trajectory, it is evident that without

this loss of pose, the overall error would have been smaller.

The *UKF17* rosbag (see here) contains a 150-second grid trajectory. Altitude changes are smooth, and turns are well defined. Position errors average $22cm$, which is acceptable. The architecture's behavior during normal operation has been validated. It is also worth noting that the UKF's pose estimation is slightly more accurate than that of the VIO.

6.4 Robustness and Blind Tracking

The goal is to evaluate the system's robustness and ensure the continuity of odometry during loss of visual tracking of the VIO.

The rosbag *UKF15_ c* records an angle variation test at $t = 4s$ and a blind turn at $t = 19s$ (see here). During the roll and pitch angle variations, there is a discrepancy of about ten centimeters between the UKF's estimate and the ground truth; this remains acceptable.

During the blind static turn, the UKF's integration of inertial data does not correctly compensate for the absence of visual features. However, a drift is observed in the X and Z directions. The altitude estimated by the UKF is accurate. Therefore, the confidence assigned (i.e., the covariance) to the IMU component in the UKF filter must be increased.

However, we note that the final poses for the UKF and OptiTrack remain very close, so the test was successful because the VIO and UKF were able to converge back to the correct position.

The *UKF22* rosbag is an example of a critical scenario (Blindness + Rotation). Starting with the second turn, the drone is blinded (see here). On the straight section, the estimated distance traveled is accurate, but the yaw angle is incorrect, and at the end of the trajectory, there is a discrepancy of approximately 40 cm.

During the blind turn, there is a massive drift in the VIO, which is corrected by the UKF. The VIO's altitude estimate Y spikes during the turn and then converges again at $t = 62$ seconds. However, while the VIO's altitude estimate returns to normal, the UKF's Z value drifts, converges during the descent to the ground, and then drifts again once on the ground. I don't know the reason for these drifts; I think the UKF's covariance matrix jumps when it retrieves values from the VIO, and this sometimes results in much too high a confidence level being placed on incorrect values.

Once the test is complete, we see that the estimate Z remains incorrect, and we observe a final offset of 40 cm. The pose "recovery" works when the camera is blind, but the error accumulates. We obtain a final RMSE for the UKF of $50cm$. Depending on the situation, this is acceptable.

The *UKF26* rosbag is a test similar to the previous one, except that this time there are multiple periods of blindness between the start and end times (see here). The results are correct for a blind test. On straightaways and in turns, the filter does not jump. However, there is a significant altitude error of about ten centimeters and an orientation error on the long straight section.

The filter thus accumulates a 40-cm error and a significant drift in the X/Z direction during static holding. The UKF's RMSE positioning error is 43 cm.

6.4.1 Summary

Position errors are high in cases where the camera is blind. However, given that the VIO is the foundation of the system, it is acceptable to have a final pose error ranging from 40 to 50 cm when it drifts. This type of visual shutdown situation is uncommon; for a quadcopter capable of traveling hundreds of meters, the error remains acceptable.

6.5 Platform Mounted on Spot

The platform is attached to the dog using 3D-printed rods (see here). These mounts are rigid. The problem is that the dog vibrates significantly when it moves, so several modifications must be made to be able to use the pose data.

Before lifting or moving the dog, the platform must be moved slightly to activate the VIO. The goal is to avoid activating the VIO by moving the dog abruptly, as this creates an initial offset between the starting pose and the activation pose—an offset that OptiTrack does not account for and that must therefore be minimized to better analyze the rest of the trajectory.

In the OpenVins configuration file (*estimator_config.yaml*), you must change *zupt_chi2_multipler* from 0.0 to 1.0. The goal is to tell OpenVins to fully correlate the camera and the IMU to indicate a stop; otherwise, the speed is not stabilized at 0 and the odometry drifts.

We multiply the noise threshold values for the *RealSense* accelerometer by 10. This amplifies the IMU's typical noise level, so that during movement, sudden and rapid accelerations caused by the motors will be ignored. We do the same for the *OlixSense* in the *imu_sync_component* file.

Next, we change the *odom0_pose_rejection_threshold* of the UKF from 1 to 5. If the VIO suddenly drifts due to a sudden movement upon startup or an impact, the fusion filter will not drift unless the pose is greater than the threshold of 5 (this unit is not in meters but is a ratio value used as a threshold). You must also change the confidence covariance of the velocities V_x and V_y to 0.49 because the covariance of the velocity topics published by *Holybro* increases slightly; therefore, you must correlate the received velocity values with other topics to avoid implementing incorrect values.

In the *spot* rosbag (see here), the trajectory is simple. However, as shown in the RMSE error analysis image, the VIO's RMSE error is $11cm$, while the UKF's is $40cm$.

When the robot is turning or moving in a straight line, the rear of the robot moves much more than the front. In this image (see here), we can see that the Holybro is mounted on an aluminum bar at the rear of the robot. This is where the problem lies. It is not possible to estimate a precise transformation matrix between the Holybro and the platform, and the results obtained are therefore not correctly integrated into the UKF. The platform is attached to the front of the robot, which explains the smaller error for the VIO.

However, even though the error is high, the robot's start and end positions do match those provided by OptiTrack, so the filter isn't bad—it just needs to be adapted to the robotic dog.

7 Conclusion

The project's objective of designing a sensor platform system capable of estimating position and orientation has been achieved. The system determines the platform's 3D position with an accuracy of about ten centimeters.

However, the constraints and requirements identified through functional and organizational analysis served as the project specifications, and, in light of these requirements, the overall outcome of the project is average. The integration of the InEKF filter and various VIO or SLAM algorithms could not be completed due to delays accumulated during the calibration of the OlixVision. Furthermore, it would have been preferable to collect more datasets for quantitative analysis.

We were nevertheless able to draw several conclusions from the research:

- Designing a modular platform allows for greater flexibility and versatility when replacing sensors and changing configurations.
- The calibration steps must be performed using open-source tools from GitHub. Don't hesitate to test multiple sensors.
- The OpenVins installation guide is accurate to within $10cm$.
- Data fusion does not always improve the VIO estimate but makes it more robust against external disturbances.
- The UKF and EKF have equivalent accuracy, but the UKF is more robust against large dynamic variations.
- Overall accuracy depends largely on the initial motion, which can cause the EKF/UKF filters to drift; the average accuracy is at least $10cm$.
- Our system maintains the same accuracy over short distances, with average errors of up to $40cm$ in the worst-case scenarios involving loss of line-of-sight and rotations.
- Selecting the VIO parameters or fusion filter settings requires extensive testing and must be done one by one.
- Changing the drone requires adjusting the pose estimation algorithm parameters due to vibrations or sensor placement.

The final product is structured as a ROS2 pipeline that retrieves trajectory datasets directly usable by researchers via *evo* [5]. It is intended to be reused, modified, and improved. The practical exercises and code are documented in various user manuals and ReadMe files. All code is available on GitHub at the following address: `https://github.com/timm-clv/SensorFusion`.

7.1 Compliance with Specifications

The summary table 7.1 confirms that the objectives for collecting visual-inertial data and benchmarking fusion algorithms were partially achieved:

Ref.	Function (Expected criterion)	Result obtained	Status
FP1	Provide the 6D position of the platform to the user Ease of acquisition	Orientation, position, and installation data are synchronized and can be executed with two commands	**Achieved**
FC1	Provide a functional and documented ROS2 architecture	Public GitHub repository provided, with comments and documentation, including a report and a README	**Achieved**
FC2	Design platforms to support all sensors and be versatile	2 modular 3D-printed platforms designed for testing and drones	**Achieved**
FC3	Use the provided sensors and calibrate them	OlixVision replaced by a RealSense D435i	**Partially Achieved** Using an other sensor is not an issue
FC4	Record and analyze multiple tests	12 rosbags provided and analyzed	**Achieved** In hindsight,20 would have been better.
FC5	Comparison against ground truth data of OptiTrack	10cm average error for the VIO in a simple case; the same applies to the UKF/EKF, but it is much more robust; 40cm average error for the UKF when the camera is blind	**Achieved**
FC6	Implement multiples VIO and fusion algorithms	Implementation of OpenVins, and robot_localisation filter EKF and UKF	**Partially Achieved** Another VIO is missing

Table 7.1: Assessment of Compliance with the Functional Specifications

7.2 Improvements

With a view toward continuous improvement and refinement of the rendering, the following options may be considered in the future:

- Reduce the error values of the intrinsic and extrinsic calibrations of the cameras and IMUs, as well as perform an extrinsic calibration between the camera, IMU, and Holybro; then inspect the results to determine whether the sensors need to be replaced.

- Integrate additional pose estimation algorithms, such as VINS-Fusion or Isaac_Ros_Visual_Slam, to compare CPU/GPU power consumption and the accuracy of odometry topics.
- Conduct more tests; with about ten rosbags per scenario, the analysis can be thoroughly comprehensive and quantitative.
- Retrieve the platform's trajectory in real time to analyze it on a third-party computer. Similar to how OptiTrack works, it is possible to publish a pose topic estimated by the Jetson over the Wi-Fi network.

7.3 Opening

There are two projects that build on our previous work: the development of an InEKF filter and a GPS module.

Implementing another fusion filter, such as the InEKF, opens the door to a scientific publication, as no previous work has mentioned the use of such a filter with two pose estimation filters (VIO + Holybro with altitude correction). Creating a package that would allow the InEKF to be configured via a YAML file would also be a major advantage, since in most scientific publications this is done directly in a *C++* or *Python* file.

Implement a GPS system on the platform. By adding another pose topic to the filters (already possible with the current filters), it will be possible to obtain a nearly perfect pose of the drone when it is connected to GPS. And when the GPS pose is no longer received, VIO and the fusion filter will ensure that the position does not diverge.

Glossary of Abbreviations and Technical Terms

AHRS — Attitude and Heading Reference System: a system that provides the 3D orientation (roll, pitch, yaw) of a platform. 20

APE — Absolute Pose Error: the absolute pose error between an estimated trajectory and a reference trajectory, after alignment. 28

CAD — Computer-Aided Design: computer-aided design. 19

CAN — Controller Area Network : bus de communication série robuste ; DroneCAN est un protocole applicatif utilisé en robotique/drones. 17

DDS — Data Distribution Service: real-time communication middleware on which ROS2 is based. 23

EKF – - Extended Kalman Filter: an extended Kalman filter, a data fusion algorithm for nonlinear systems based on local linearization (Jacobians). 7

FPC — Flexible Printed Circuit: a flexible printed circuit. 22

GPS — Global Positioning System: satellite-based positioning system. 4

IMU — Inertial Measurement Unit: inertial measurement unit that measures accelerations and angular velocities. 7

InEKF — Invariant Extended Kalman Filter: a variant of the EKF that exploits the properties of Lie groups to achieve greater robustness against nonlinearities. 7

MSCKF – - Multi-State Constraint Kalman Filter: a Kalman filter with multi-state constraints, used in particular in VIO architectures. 7

QoS — Quality of Service: a policy governing the reliability of ROS2 message transmission (e.g., Reliable, Best Effort). 25

REP-105 — ROS Enhancement Proposal 105: ROS standard defining naming conventions for coordinate reference frames (base_link, odom, map). 20

RMSE — Root Mean Square Error: a metric that heavily penalizes large errors. 28

ROS — Robot Operating System: middleware for the development of robotic applications. 4

SLAM — Simultaneous Localization and Mapping: simultaneous mapping and localization of an autonomous system in an unknown environment. 7

SSH — Secure Shell: secure remote connection protocol. 49

TF2 — ROS2 library for managing geometric transformations between coordinate frames. 12

TOPS — Tera Operations Per Second: unit of measurement for computing power. 14

UKF — Unscented Kalman Filter: a nonlinear alternative to the EKF that uses sigma points rather than linearization. 7

URDF — Unified Robot Description Format / XML Macro: formats for describing a robot's geometry; XACRO allows parameterization via XML macros. 8

VIO — Visual-Inertial Odometry: visual-inertial odometry, pose estimation via camera + IMU fusion. 6

YAML – - YAML Ain't Markup Language: a structured configuration file format used for ROS2 parameters. 8

ZUPT — Zero Velocity Update: a technique that sets velocity to zero when the system is detected to be stationary, in order to limit drift. 28

Bibliography

[1] CRTA web site, 2026 version. https://crta-robotics.com.

[2] CRTA-Lab. (2024). stage_ros2. [GitHub Repository]. Available at : https://github.com/CRTA-Lab/stage_ros2.

[3] PX4 Autopilot Open Source Project, Optical Flow Kinematic Compensation. Available at : https://docs.px4.io/main/en/sensor/optical_flow.

[4] P. Geneva. (2023). Visual-Inertial Sensor Calibration – A Complete Tutorial and Discussion. [Online video]. Available at : https://www.youtube.com/watch?v=BtzmsuJemgI. P. Geneva. (2023). Visual-Inertial Sensor Calibration – Live OpenVINS State Estimation Demo. [Online video]. Available at : https://www.youtube.com/watch?v=rBT5O5TEOV4

[5] M. Grupp. evo. [GitHub repository]. Available at: https://github.com/michaelgrupp/evo

[6] Autonomous Systems Lab (ETHZ). kalibr. [GitHub repository]. Available at: https://github.com/ethz-asl/kalibr

[7] Oxford Robotics Institute (DRS). allan_variance_ros. [GitHub repository]. Available at: https://github.com/ori-drs/allan_variance_ros

[8] Robot Perception and Navigation Group (RPNG). open_vins. [GitHub repository]. Available at: https://github.com/rpng/open_vins

[9] NVIDIA. isaac_ros_visual_slam. [Online documentation]. Available at: https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_visual_slam/isaac_ros_visual_slam/index.html

[10] Charles River Analytics (CRA). robot_localization (branche rolling-devel). [GitHub repository]. Available at: https://github.com/cra-ros-pkg/robot_localization/tree/rolling-devel

[11] M. Pilté, S. Bonnabel, F. Barbaresco (2017). Drone Tracking Using an Innovative UKF. Mines Paris - PSL (HAL Open Archive). [Scientific publication]. Available at: https://minesparis-psl.hal.science/hal-01692469v1

[12] S. Boylan, C. Jiang, G. Haggin, J. Wang, X. Zhao. University of Michigan. (2020). 1 Left-Invariant EKF for Robot Localization : A Mobile Robotics Project. [Online documentation and GitHub repository]. Available at: https://github.com/ghaggin/invariant-ekf/tree/master

[13] Boston Dynamics (2026). Spot - The Agile Mobile Robot. [Specifications Page]. Available at: https://bostondynamics.com/products/spot/

8 Annexe

These appendices are intended to help readers explore certain concepts discussed in the previous chapters in greater depth.

8.1 GitHub

All of my work is freely available and hosted on GitHub. You can find it by clicking here.

8.2 Internship Informations

The documents are administrative PDF files from my school and are written in French. They are available on the following page. (Back to introduction chapter 1)

Intégrité scientifique et académique

Engagement de non plagiat

Je soussigné(e), Timm Claverie.
N° carte d'étudiant : 22200303.
Inscrit(e) en deuxième année d'école d'ingénieurs parcours SYSMER.

Déclare avoir pris connaissance du règlement général des études et notamment de la section 7.3 au plagiat. Je suis pleinement conscient(e) que la copie intégrale sans citation ni référence de documents ou d'une partie de document publiés sous quelques formes que ce soit (ouvrages, publications, rapports d'étudiant, internet etc…) est un plagiat et constitue une violation des droits d'auteur ainsi qu'une fraude caractérisée. En conséquence, je m'engage à citer toutes les sources que j'ai utilisées pour produire et écrire ce document.

Usage de l'IA

L'objectif de cette déclaration est de favoriser la transparence et de développer votre responsabilité professionnelle face aux outils numériques. Elle vise à vous sensibiliser aux implications pédagogiques de l'IA tout en permettant aux formateurs d'évaluer votre capacité à l'utiliser de façon pertinente et responsable.

☐ Je n'ai utilisé aucun outil d'IA pour la réalisation de ce rapport
☒ J'ai utilisé un / des outil(s) d'IA pour la réalisation de ce rapport
Outil(s) IA utilisé(s) : Gemini, Claude, Chat GPT.
☒ 1. J'ai utilisé un outil d'IA générative pour améliorer la rédaction (orthographe, grammaire, syntaxe) du document que j'ai rédigé
☐ 2. J'ai utilisé un outil d'IA pour structurer mon travail, m'aider à définir le plan du travail.
☒ 3. J'ai utilisé un outil d'IA pour traduire des textes inclus ou non inclus dans mon travail.
☒ 4. J'ai utilisé un outil d'IA générative comme moteur de recherche pour explorer, simplifier ou identifier des références pertinentes (sans intégration directe).
☒ 5. J'ai utilisé un outil d'IA générative pour obtenir des idées ou des contenus que j'ai ensuite retravaillés et intégrés dans mon travail.
☐ 6. Si j'ai utilisé des phrases entièrement générées par l'IA je l'ai mentionné comme une référence
☐ 7. J'ai conservé les prompts et les échanges réalisés avec l'IA et je les tiens à la disposition du correcteur.

Je certifie que l’utilisation de l'IA dans le cadre de ce travail a été effectuée dans le respect des règles d’intégrité académique.

Fait le 18 août 2026 à Toulon
Signature

8.3 Project Management and Functional Analysis

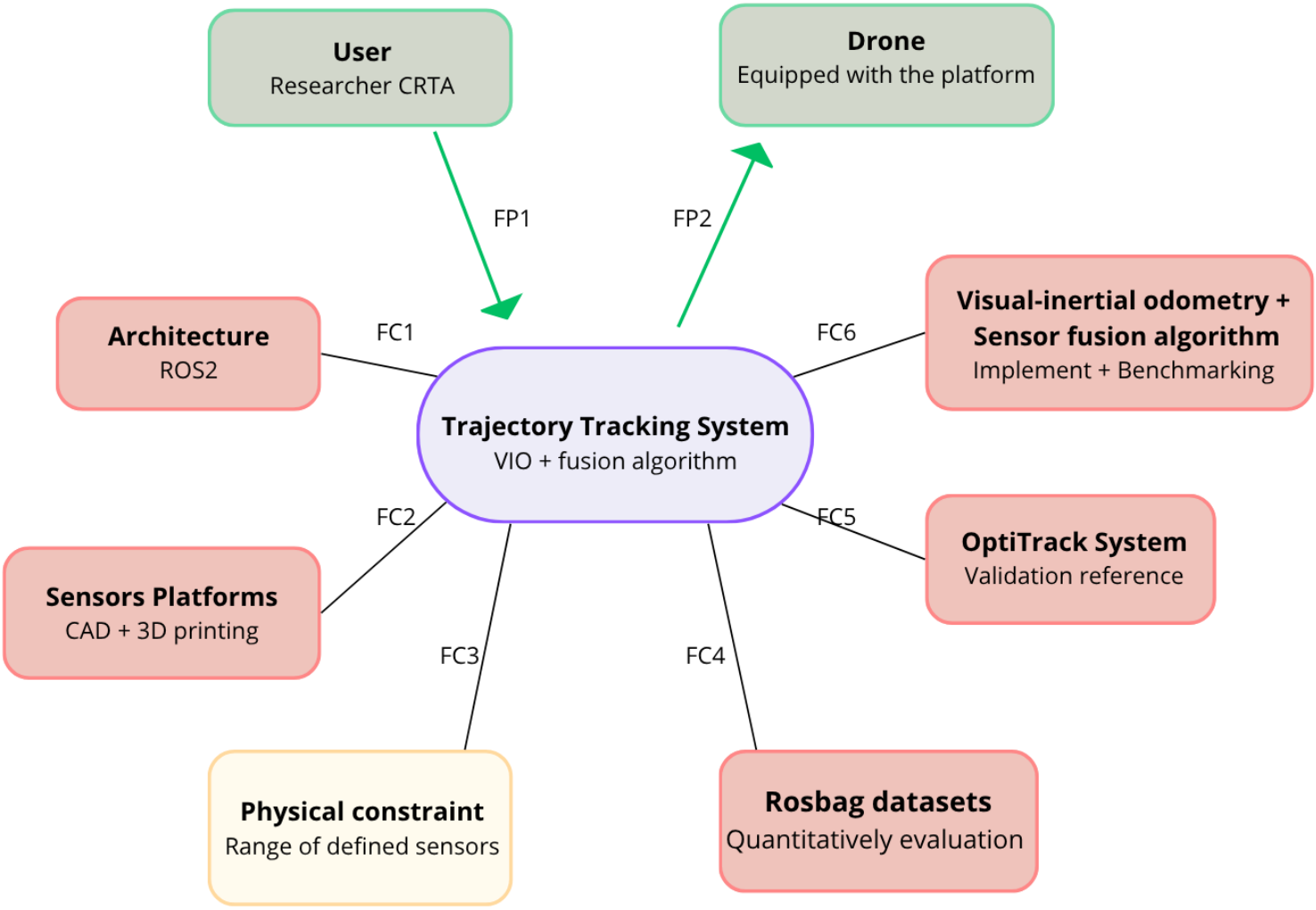


Figure 8.1: Ocotopus Diagram

The information in this table is discussed later in the report. The purpose of the figure is to provide a quick and concise overview of the project's key issues. (Back to report 1.2.2).

8.4 Tree structure

To return to the report: 3

```
sensor_platform_ws/
└── src/
    ├── hardware/
    │   ├── realsense-ros (Intel RealSense driver)
    │   │   └── Step 3: Clone the official repositories and ensure they compile
    │   │                for ROS2 Humble.
    │   └── uavcan_bridge (CAN bus node - Holybro H-Flow)
    │       └── Phase 3: Since the Holybro sensor communicates via the DroneCAN protocol,
    │                    configure the hardware interface (Jetson CAN bus or USB/CAN
    │                    adapter). Write or integrate a node that translates DroneCAN
    │                    frames into ROS2 geometry_msgs/TwistWithCovarianceStamped
    │                    messages.
    │
    ├── olive_networking/ (udev rules, IP scripts, and DDS XML)
    │   ├── Phase 2 (Script): Write a bash script that assigns distinct static IP addresses
    │   │                      for the camera and the IMU) via the Ethernet-over-USB
    │   │                     interface.
    │   ├── Phase 2 (udev)   : Create .rules files to physically map the Jetson's USB ports
    │   │                      to predictable network interface names.
    │   └── Phase 2 (DDS)    : Create an XML file (FastDDS or CycloneDDS) to optimize
    │                          the discovery of Olive nodes on the local network and ensure
    │                          real-time communications.
    │
    ├── platform_description/ (URDF/XACRO files - compliant with REP-105)
    │   └── Phase 4 (XACRO/URDF): Model the robot's structure. Define the base_link
    │                             (robot's center) and reate the exact static
    │                             transformations between base_link and each sensor,
    │                             strictly adhering to the axes specified in the
    │                             documentation.
    │
    ├── platform_bringup/ (Launch files and .yaml parameters)
    │   ├── Phase 3 (Drivers)    : Configure the Xsens and RealSense driver to disable
    │   │                          unnecessary features (CPU power saving).
    │   ├── Phase 5 (Bringup)    : Create the master file system.launch.py that starts the
    │   │                          architecture (network, URDF, hardware, fusion nodes).
    │   └── Phase 6 (Validation): Record rosbags with the platform in motion.
    │                             Compare the fused odometry with the OptiTrack data.
    │
    └── platform_processing/ (C++ components, synchronization, and Isaac ROS)
        └── Phase 1 (test): Create a script that subscribe and publish topics of
             Olix sensors
```

8.5 Overview of Equipment

Click here to view the datasheets for the various sensors. The D435i camera datasheet is not available on the drive because it was not needed during the project. (Back to report [3])

8.5.1 Connection to Jetson

I connect to the mini-computer via SSH.
At first, the wired connection (a USB cable between my PC and the Jetson) was convenient and allowed me to easily test the sensors' USB connections.

```
ssh darts@192.168.55.1
```

This command is run in my PC's terminal; the IP address is obtained by scanning the PC's USB connections. As for the name, it was given to me. Once this command has been run, you need to enter the Jetson's password, which is *crta*.

After that, I opted to use *SSH* connections via Wi-Fi IP to test the sensor platform while on the move. The IP address for this connection therefore varies depending on the Wi-Fi network. Here's how to configure your connection for your Wi-Fi network :

Connect to the Jetson via a wired connection, then run :

```
nmcli device wifi list
# or just :
nmcli device
```

This will display the various connected devices. Next, find the name of the *device* whose type is *wifi*, and then enter the command :

```
sudo nmcli device wifi connect "name of the device" password "the wifi password"
ip -4 addr show "name of the device"
```

You have now changed your Wi-Fi settings and can connect to your Jetson wirelessly using your new IP address. This address is displayed in the output of the last command; it is the IP address that follows the word "inet" (which is simply an abbreviation for "Internet" and refers here to a family of communication protocols).

(Back to report [3.1.1])

8.5.2 Display Olive Robotics Sensors

On these various pages, the settings we're most interested in are at the bottom right: they are the *Network Settings* and *ROS2 Settings*. (Back to report [3.1.2])

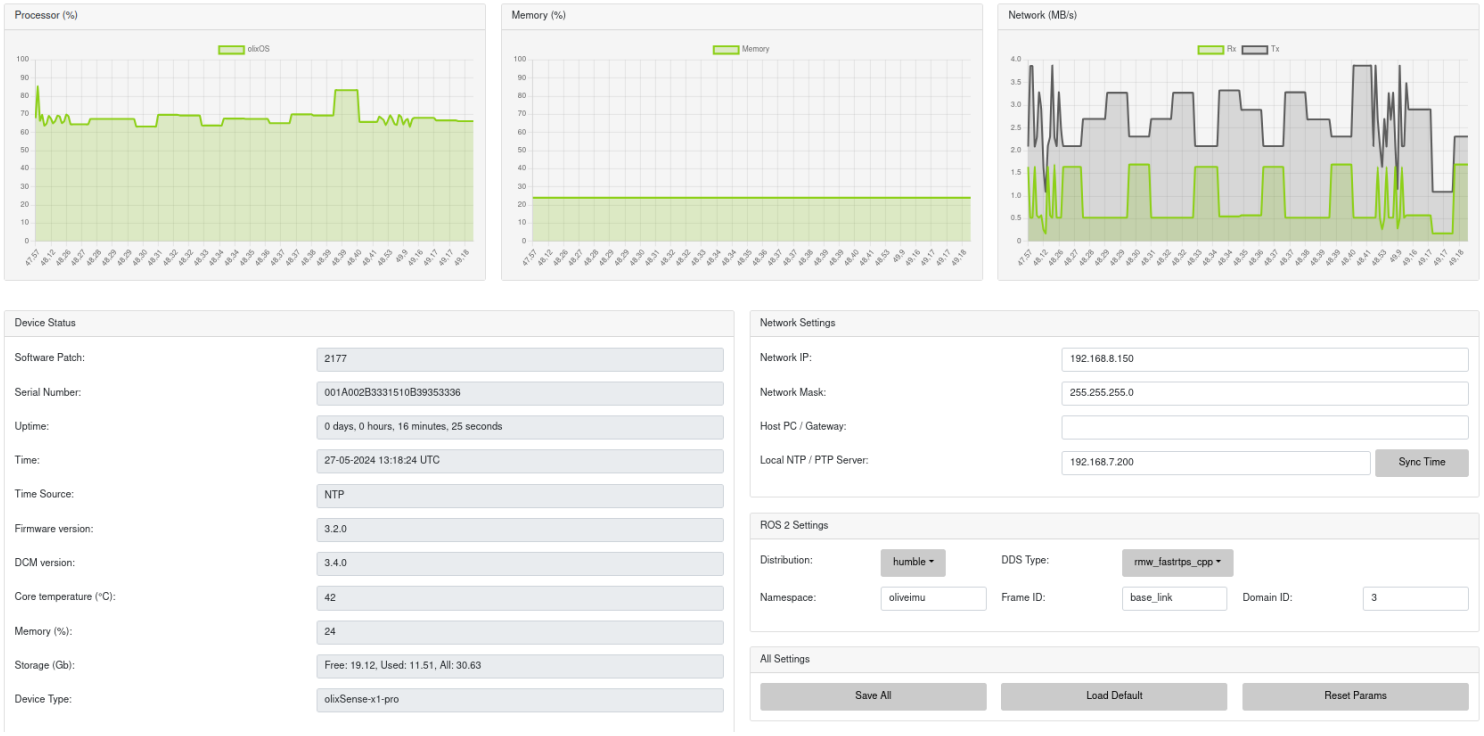


Figure 8.2: Local dashboard, at http://192.168.8.150/, of the IMU

8.5.3 Getting Started H-Flow

In this section, you'll find the key points for using H-Flow. (Back to report [3.1.4])

The serial-CAN link is initialized at 1 Mbps using the following commands:

```
sudo apt install can-utils
# Cleaning up cumbersome legacy processes
sudo pkill slcand
sudo slcand -o -s8 -t hw -S 3000000 /dev/ttyACM0 slcan0
# Software activation of the network interface
sudo ip link set up slcan0
```

We have *-o*, which opens the device; *-s8*, which sets the CAN bus bitrate to 1 Mbps; and *-t hw*, which prevents high-frequency frame loss. , *-S 3000000*, which sets the USB serial communication speed to 3 Mbps to avoid a bottleneck, */dev/ttyACM0*, which is the name of the adapter for the Jetson, and finally *slcan0*, which is the name assigned to the device.

The proper functioning of DroneCAN traffic cam be verified using the command `candump slcan0`.

Development of the C++ Bridge Node

The `uavcan_bridge` package is responsible for injecting these frames into ROS2. The main component `dronecan_bridge_component.cpp` integrates the low-level open-source library `libcanard`. The libcanard library retrieves data from a 16 KB static memory pool. Normally, memory allocation is dynamic, but this has been replaced to avoid the risk of real-time memory failures :

```
... uint8_t canard_memory_pool_[16384];
```

This node listens for **Message ID 20200** corresponding to the DroneCAN structure `com.hex.equipment.flow.Measurement` and converts the variables into ROS2 messages of type `geometry_msgs/TwistWithCovarianceStamped`, published on the topic `/optical_flow/velocity`.

Workaround for Dynamic Node ID Allocation

Since the DroneCAN protocol requires that a device have a Node ID to transmit, and the Holybro sensor requires a dynamic ID allocator (not natively implemented in the C++ node), a workaround was implemented. A Python virtual environment was created on Ubuntu 22.04 to run the tool:

```
cd ~/sensor_platform_ws
python3 -m venv dronecan_env
source dronecan_env/bin/activate
pip install dronecan-gui-tool
dronecan_gui_tool
```

It acts as an allocation server and dynamically assigns **Node ID 125** to the Holybro sensor, which instantly enables the transmission of its data frames, as shown in the figure 8.3.

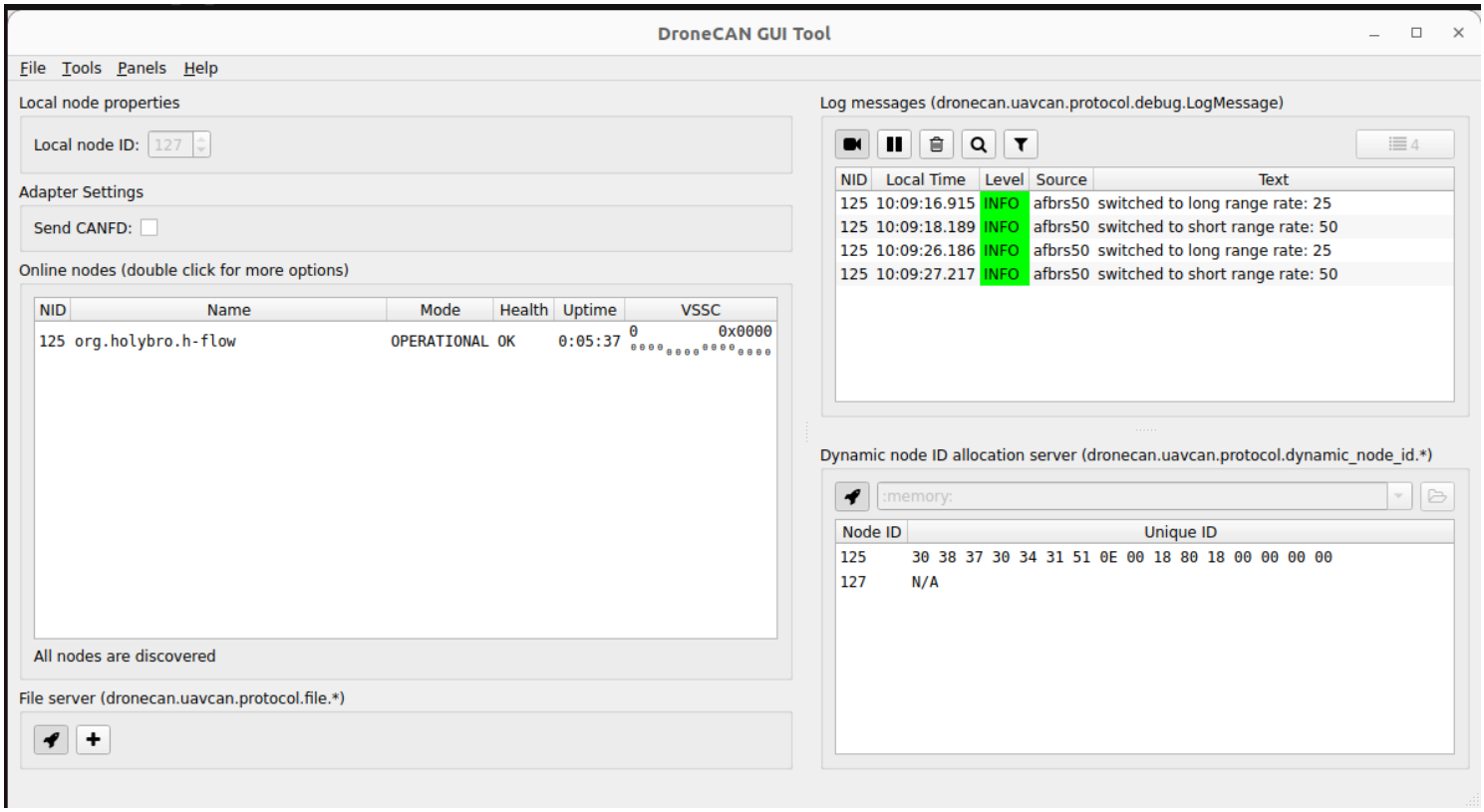


Figure 8.3: IH-Flow on GUI Tool Interface

(You can see here that the node ID is 127, because the H-flow has been unplug and plug 2 times before.) And this is where we retrieve the names of the shared data

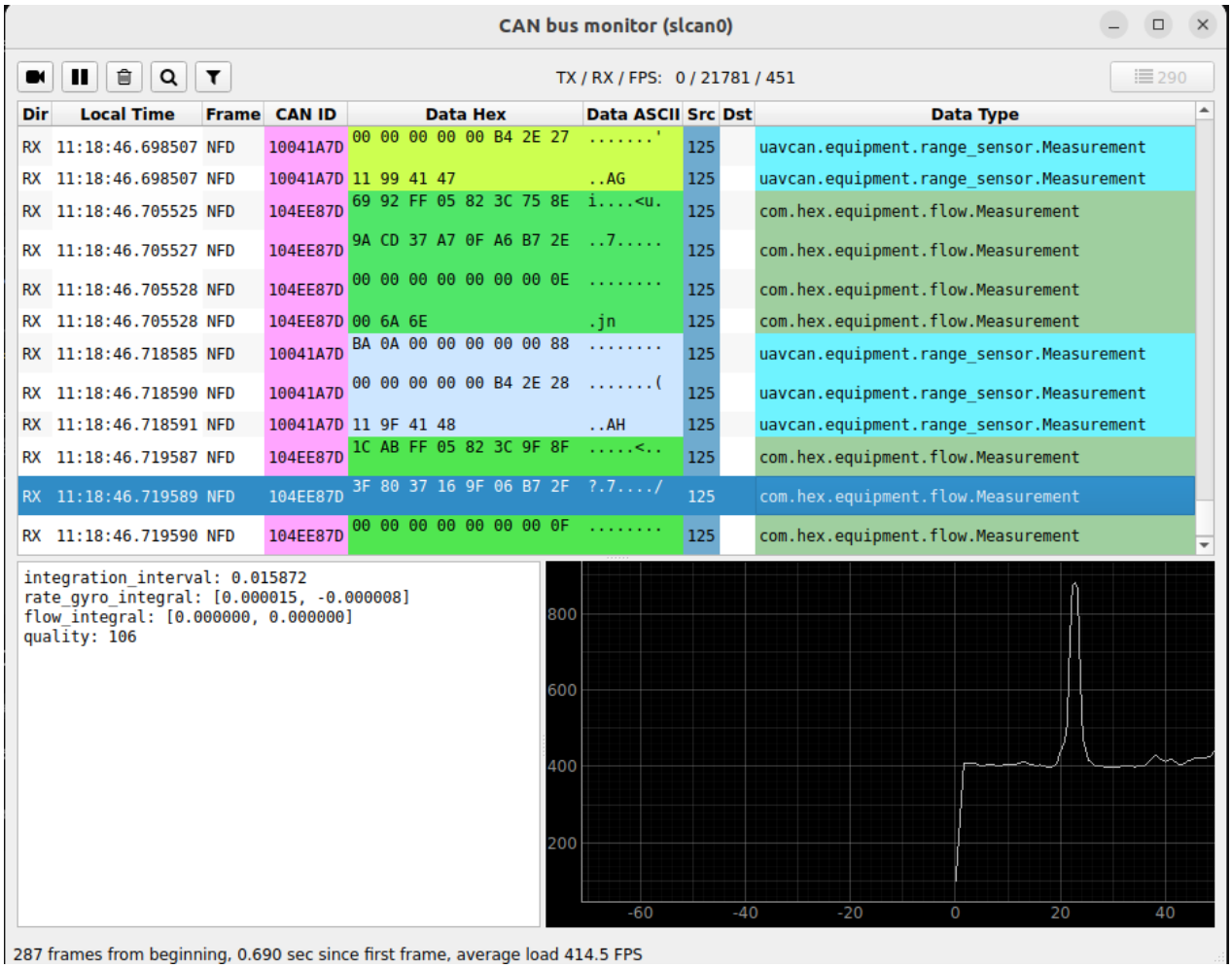


Figure 8.4: Data flow on GUI Tool Interface

The *rate_gyro_integral* value provides information about the H-flow's tilt (roll and pitch). The *flow_integral* refers to the value of the beam's edge change in the X and Y directions. And *quality* describes the roughness of the ground. On a plastic-coated surface, this value is around 80, compared to 120 on concrete.

Velocity topic

The formula for constructing velocity topics is directly inspired by the PX4 Autopilot. This is onboard software for drones that works with multiple EKFs. Integrating it into our architecture solely to control our *Holybro* would have been too cumbersome, and it might not have been possible to derive a ROS2 topic from it.

The linear velocity formula is expressed as:

- $flow\_integral\_x/integration\_interval$: this is the angular rate (rad/s); that is, the raw optical flux is converted to angular velocity by dividing the accumulated angle by the integration time.
- $gyroscope\_rate\_x$: this is the de-rotation. The gyroscope provides the actual angular velocity of the sensor about the corresponding axis. By subtracting this value, the purely rotational contribution to the measured flux is removed, leaving only the component due to translation.

- ground_distance: angle-to-linear-velocity conversion. For a point on the ground at a distance d, the relationship between angular velocity and linear velocity is: $V \approx \omega \times d$. Therefore, multiplying the de-rotated angular rate by the ground distance yields the actual linear velocity.
- I noticed that the results were significantly better when $gyroscope\_rate/integration\_interval$, perhaps because the gyroscopic flux must be segmented upstream in the sensor.

Thus, the equation is: (measured angular rate - angular rate due to sensor rotation) × distance = pure translational velocity. This is exactly the same principle as that used by conventional optical flow sensors such as the PX4 (PMW3901 + gyro): $V = (flow\_rate - gyro\_rate) \times height$.

To perform a qualitative analysis of the flow, I compared the linear velocities in the x and y directions with those published by the VIO, which will serve as the ground truth.

There is a tremendous amount of noise; I think *PX4* uses a noise-smoothing algorithm for the gyro and the optical flux. I tried applying this filter directly to these data streams, but it didn't improve the results. I did, however, apply a low-pass filter to the linear velocities and the z distance to limit overall variations over short periods (noise).

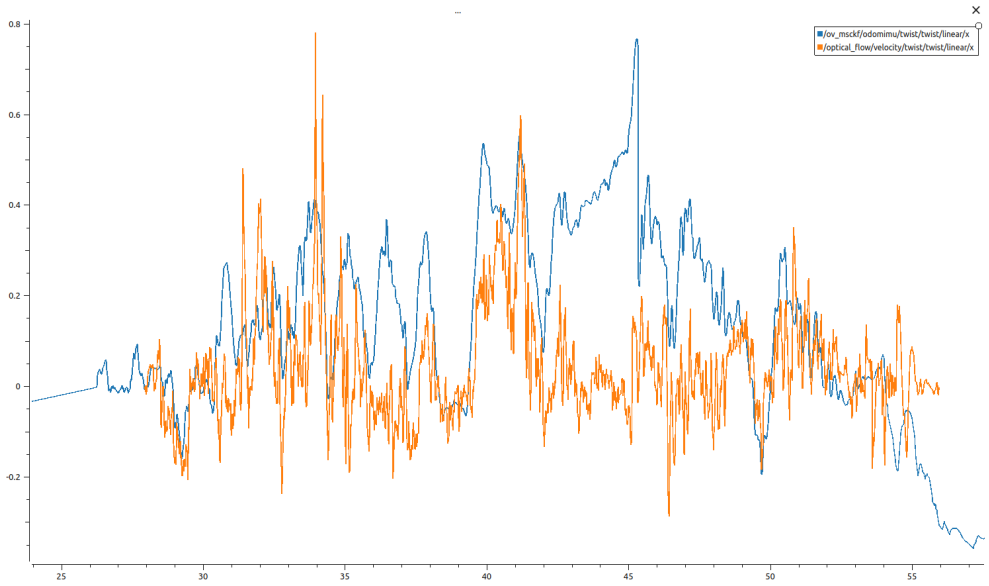


Figure 8.5: Linear speed on x axis estimated by optical flow

In Figure 8.5, the blue curve represents the ground truth, and the orange curve the *Holybro*'s linear velocity estimate. The orange curve diverges from the blue curve on two occasions. These dates correspond exactly to the turning period (on-the-spot rotation of the platform); the camera located at the end of the platform travels a greater distance than the *Holybro*, which explains these discrepancies. The results are similar for the V_y graph.

I consider these initial linear velocity estimates to be sufficient. Even if they are not suitable for static turning situations, this is not a problem because, during sensor fusion, the IMU will take over by compensating for this lack of information with its yaw angle and gyroscopic estimates.

(Back to report [3.1.4])

8.5.4 Sensors Calibration

In this subsection, I discuss in detail how to calibrate a sensor and analyze a calibration report. All image-to-camera and IMU-to-camera calibrations were performed using *Kalibr* [6]. I used Patrick Geneva's videos as the basis for my tutorial, even though he wasn't working with ROS2 [4].The IMU calibrations (only) were performed using *allan_variance_ros* [7] . (Back to report [3.2])

OlixSense

The purpose of calibrating the OlixSense IMU is to determine two error metrics for the gyroscope and the accelerometer. These are the *noise_density*, which examines short-term fluctuations per frequency, and the *random_walk*, which tracks long-term drift.

The larger the dataset, the better the IMU calibration will be. Place the IMU on a slightly flat surface for 24 hours or more and start recording.

Note: The IMU must not be completely flat or in its initial position; otherwise, all pose and orientation values may be zero, and the calibration will not work.

Note: It is also important to save the rosbag as a *.mcap* file; if it is saved as a *.db3* file, it will not work. Here is an example command to save the rosbag:

```
ros2 bag record -s mcap /olive/olixSense/x1/oliveimu/imu -o dataset_cam3
```

Once the rosbag has been saved, you can begin the analysis. This process can take several hours and depends mainly on the *config.yaml* file. In this file, you specify the topic to be analyzed, the duration of the rosbag, its publication frequency, and the frequency at which it should be analyzed. The lower this value is (starting from a reasonable threshold, of course), the longer the calibration takes, but the more accurate it is.

```
cd ~/SensorFusion/aw_ws
colcon build
source install/setup.bash
SO_DIR=$(dirname $(find ~/SensorFusion/aw_ws -name "liballan_variance_ros2.so" | head -n
    1))
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$SO_DIR
ros2 run allan_variance_ros2 allan_variance ./dataset_imu ./config.yaml
```

Once the code has compiled, a *.csv* file was generated. By running the following command, you will be able to obtain the calibration files.

```
python3 ./src/allan_variance_ros2/src/allan_variance_ros2/scripts/analysis.py --data ./
   data/output2/allan_variance.csv
```

The two graphs generated allow you to analyze the quality of your IMU and observe its drift. We will focus on the *imu.yaml* file. You must multiply the *noise_density* values by 5 and the *random_walk* values by 10 because *allan_variance_ros* underestimates these values compared to reality: By increasing these covariance values, we reduce our reliance on the sensors and limit our margin of error. The calibration results are available here .

OlixVision

This section discusses various types of intrinsic and extrinsic calibrations in detail.

For intrinsic calibration :

1. We save the */olive/camera/image_raw* topic—generated by *decompressor.py* (see 8.8.1) from the */olive/camera/olivecam/image/compressed* topic containing the compressed camera image—to a rosbag.
2. We move the camera around in front of the Arucos checkerboard; the camera must have seen Arucos markers across its entire image plane to achieve proper calibration. Therefore, it is essential to vary the distance between the camera and the checkerboard, vary the camera's yaw angle to create perspective, and finally vary its roll angle.
3. Our computer uses *ROS2*, and Kalibr is designed for *ROS1*. Therefore, we must convert the rosbag2 file to rosbag1. To do this, we use a command that creates a binary file compatible with the ROS1 language (see 8.8.2).

In the *kalibr* folder, start by enabling graphical display for Kalibr windows.

```
xhost +local:root
```

Next, navigate to the *data_calib* folder. Place the final rosbag file there (the one containing the RAW images and the .bag file). Then, enter the folder and launch the kalibr environment:

```
cd /data\_calib
docker run -it -e "DISPLAY" -e "QT_X11_NO_MITSHM=1" \
    -v "/tmp/.X11-unix:/tmp/.X11-unix:rw" \
    -v "$(pwd):/data" kalibr:20.04
```

Once you reach this level, you need to source the environment and navigate to the *data* folder (which is hidden); this will force the environment to close without losing the analysis of your files. These files will appear directly in the rosbag folder located in *data_calib*.

```
cd /data
source /catkin_ws/devel/setup.bash
export KALIBR\_BIN=/catkin\_ws/devel/lib/kalibr
```

Now, depending on whether the camera has a rolling shutter, you may or may not need to include the *_rs_* tag. You need to create a target.yaml file where the test chart measurements are listed. Then, you can start using commands to analyze your rosbags.

```
cd calib_dataset16_raw_ros2
/catkin_ws/devel/lib/kalibr/kalibr_calibrate_rs_cameras \
    --target /data/target.yaml \
    --bag calib_dataset16_raw_ros1.bag
    --model pinhole-equi-rs \
    --topic /olive/camera/image_raw \
    --frame-rate 18 \
    --inverse-feature-variance 1
```

You will be prompted to initialize the focal length values; personally, I entered *400 400*. Once the compilation is complete, you will get a file named *calib_dataset16_raw_ros1-imu.yaml*. Please move it out of the folder

and rename it to *cam.yaml*.

You can now proceed to the **extrinsic calibration** of the camera-IMU.

When registering the rosbag, you must:

1. Unpack the camera topic and set the rosbag to *ROS1*.
2. Move the camera enough to trigger the IMU; the more it is triggered, the better the calibration will be.
3. Be careful not to move the camera too much because, since it uses a rolling shutter, if the camera moves too much, a pixel shift occurs in the image, which increases the Arucos reprojection error.

For the *OlixVision*, I tested two types of IMUs. The first calibration was performed between the camera and the *OlixSense* IMU (not its onboard IMU); the results were poor because, depending on the distance between the camera and the IMU, a lever arm effect occurs and the resulting transformation matrix is incorrect. Then I attempted a second calibration using the camera's internal IMU; in this case, I obtained good results—they weren't perfect, but they were acceptable.

Here is the command to start the calibration. First, you must have the *cam.yaml, imu.yaml, and target.yaml* files in the same folder. During testing, the calibration was performed between the topics */olive/camera/olivecam/filtered_ahrs* and */olive/camera/image_raw*.

```
cd kalibr/data\_calib
cd calib_dataset17_raw_ros2
/catkin_ws/devel/lib/kalibr/kalibr\_calibrate_imu_camera     --imu-models scale-
    misalignment    --reprojection-sigma 1.0    --target /data/target.yaml    --cam /data
    /cam.yaml    --imu /data/imu.yaml    --bag /data/calib_dataset17_raw_ros1.bag   --
    show-extraction
```

Several files are generated; the most important ones are: *calib_dataset17_raw_ros1-camchain-imucam.yaml, calib_dataset17_raw_ros1-imu.yaml, and calib_dataset17_raw_ros1-report-imucam.pdf*. The first file contains data related to the camera, including its adjusted intrinsic parameters, distortion model, and transformation matrix. The second file is similar, except that it pertains to the IMU; it includes covariance matrices for the gyroscopes and accelerometers. Finally, the last file is the overall report, which allows us to assess the quality of the calibration.

I would also like to point out that, like *OlixSense*, the camera's internal IMU has been calibrated using *allan_variance_ros*.

Report Analysis:

The camera error is low: *1.92*, as are the errors for the gyroscope (*1.12*) and the accelerometer (*0.66*). However, on page 5 of the report, we observe that the temporal jump variance of the topics is high *±1.7671*; on page 12, the camera's reprojection errors for the Arucos are greater than 1 pixel in radius—whereas, for optimal performance, they should be less than that—but the pose estimates provided by the IMU are correct. The differents calibration results are available here .

D435i

The procedure for calibrating the *D435i* is somewhat similar to that of the *OlixVision*. However, we are working with two infrared cameras operating in stereo mode without *Rolling Shutter*.

Unlike the previous camera, there are no restrictions on the camera's frame rate when recording rosbag. Additionally, the camera outputs its image streams natively in RAW format, so no decompressor is needed. Here is the command for calibrating the cameras:

```
/catkin_ws/devel/lib/kalibr/kalibr_calibrate_cameras     --target /data/target.yaml
   --bag /data/d435i_calib_ros1.bag     --models pinhole-radtan pinhole-radtan    --
   topics /d435/d435_node/infra1/image_rect_raw /d435/d435_node/infra2/image_rect_raw
     --bag-freq 10.0    --show-extraction
```

In the output file, we can see that, in addition to the intrinsic calibration, an extrinsic calibration between the two cameras was also performed. The errors obtained are minimal: `reprojection error: [-0.000015, -0.000001] ± [0.298873, 0.310946]`. You can view the report here .

The camera-IMU calibration was performed using the RealSense's onboard IMU.

```
/catkin\_ws/devel/lib/kalibr/kalibr_calibrate\_imu_cameras     --target /data/target.
   yaml    --cam /data/cam.yaml    --imu /data/imu.yaml     --bag /data/d435i\_imu\
   _calib_ros1.bag        --bag-freq 10.0    --show-extraction
```

The errors obtained are extremely small compared to the calibrations performed with the *OlixVision*. Two *.yaml* files are generated, which will be used by the VIO. You can consult these files here .

It should be noted that the variation in the arrival time of messages sent by the IMU is very small this time; see 8.6. I believe this is the main reason for the problems encountered in previous calibrations.

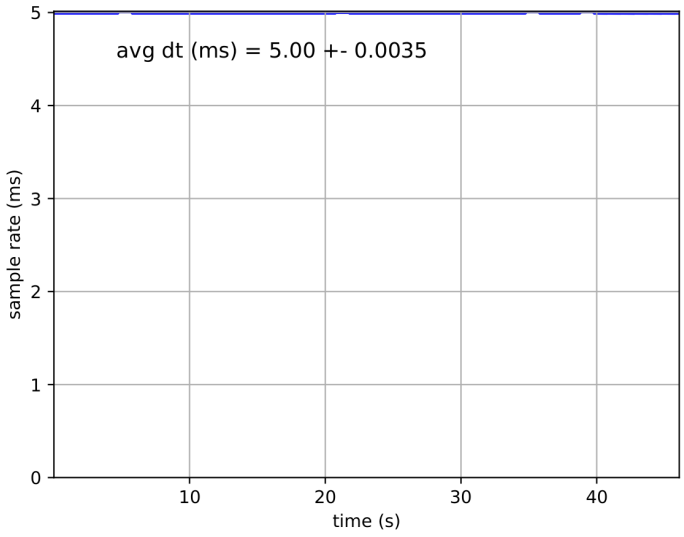


Figure 8.6: Embedded IMU, sample inertial rate

(Back to report [3.2])

8.6 Overall Rules

(Back to report [5.1])

8.6.1 Communication and ID Rules

I focused on the low-level network infrastructure required to host Olive Robotics sensors communicating via the Ethernet-over-USB protocol.

Definition of Network Roles

- `olive_network_configuration.sh` : Bash script that automates the assignment of static IP addresses to the network interface using NetworkManager `nmcli`.
- `99-olive-networking.rules`: `udev` rules file responsible for permanently associating the physical MAC address of each USB device with a name to help with debugging.
- `fastdds_profiles.xml`: An XML configuration file that restricts DDS traffic to sensor interfaces only via a whitelist of local IP addresses (interfaceWhiteList) and increases the size of UDP buffers to prevent high-frequency packet loss. The goal is to accelerate discovery and ensure strict real-time data transfers.

These scripts were run only once at the beginning of the global architecture configuration. They are not rerun each time by *system.launch.py*. It is best not to rerun them, as reassigning sensor names at each startup would likely lead to IP conflicts.

Practical Implementation and Validation

In this section, I'll go over the different steps I followed to establish the above rules.

Initially, the `ip a` command revealed unstable generic interfaces associated with dynamic IP addresses allocated by default:

```
16: enx0a575b00dfce:... link/ether 0a:57:5b:00:df:ce...
    inet 192.168.7.161/24 scope global dynamic...
```

To address this, the rules file `/etc/udev/rules.d/99-olive-networking.rules` has been configured:

```
# udev rules for Olive Robotics sensors (Ethernet-over-USB)
SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="0a:57:5b:00:df:ce", NAME="enx_olive_cam
    "
SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="a2:10:11:e2:ae:4e", NAME="enx_olive_imu
    "
```

The network subsystem was immediately hot-reloaded via :

```
sudo udevadm control --reload-rules
sudo udevadm trigger --subsystem-match=net
```

As a result, the hardware interfaces have been standardized under the following permanent names: `enx_olive_cam` et `enx_olive_imu`.

Next, the network configuration script was made executable and applied:

```
chmod +x ~/sensor_platform_ws/src/olive_networking/scripts/olive_network_configuration.
    sh
sudo ~/sensor_platform_ws/src/olive_networking/scripts/olive_network_configuration.sh
```

The terminal supports deterministic IP address assignment:

```
Starting the network configuration for the sensor platform...
-> Configuration of enx_olive_cam with IP 192.168.7.101/24...
   [OK] enx_olive_cam configured (Olive_enx_olive_cam).
-> Configuration of enx_olive_imu with IP 192.168.7.102/24...
   [OK] enx_olive_imu configured (Olive_enx_olive_imu).
================================================
Verification of interfaces:
19: enx_olive_cam: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500...
    inet 192.168.7.101/24 scope global noprefixroute enx_olive_cam
18: enx_olive_imu: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500...
    inet 192.168.7.102/24 scope global noprefixroute enx_olive_imu
================================================
```

Finally, the optimization of the DDS transport layers was recorded/established by adding this to the end of the `~/.bashrc`:

```
# ROS2 DDS Configuration - Sensor Platform
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export FASTRTPS_DEFAULT_PROFILES_FILE=~/sensor_platform_ws/src/olive_networking/config/
fastdds_profiles.xml
export ROS_DOMAIN_ID=42
```

Running `source ~/.bashrc` and checking the `$ROS_DOMAIN_ID` variable confirm that all ROS2 nodes on the platform now share an isolated domain (ID 42) and apply the required real-time QoS profiles.

8.6.2 USB Rules

I've encountered various issues with USB communications—make sure you have USB 3.0 cables with a blue stripe on the inside. The cameras need to send large amounts of data (images + IMU) over USB, so you may need to adjust the USB port settings on your Jetson.

We have *install_rules.sh*, which partially handles USB rules along with *olive_network_configuration.sh*. The following lines increase the network receive and transmit buffers:

```
sysctl -w net.core.rmem_max=8388608
sysctl -w net.core.rmem_default=8388608
sysctl -w net.core.wmem_max=8388608
sysctl -w net.core.wmem_default=8388608
```

However, by default, Ubuntu limits the USB file system (usbfs) buffer to 16 MB. This is insufficient for the raw video streams from the *D435i* and will cause USB congestion. You must increase this limit (for example, to 1000 MB):

```
echo 1000 | sudo tee /sys/module/usbcore/parameters/usbfs_memory_mb
```

(Back to report [5.1])

8.7 Workspace Architecture

8.7.1 OlixVision Data Acquisition

All data acquisition takes place in the same file as the IMU data acquisition, *imu_sync_component.cpp*. The entire code has been commented out because the camera is not being used.

We reliably retrieve the topics as follows :

- */olive/camera/olivecam/image/compressed* streams at $20Hz$ on a *best-effort* basis,
- */olive/camera/olivecam/filtered_ahrs* streams at $200Hz$ on a *best-effort* basis.

The timestamp associated with the camera is December 2022; these *timestamps* cause the EKF/UKF to crash. The purpose of the code is to retrieve the topics and republish them with the correct timestamp and the corresponding *frame_id* in the URDF.

It should be noted that the camera's IMU topic is filtered (*_ahrs*); this may be the source of the IMU bug, even after calibration, since the filter may be adaptive. I tried to work around the problem by rewriting the IMU topic using the camera's pose, twist, and linear acceleration topics, but the results were poor.

The *imu_sync_component.cpp* code must run in parallel with *decompressor.py* (see 8.8.1) in order to produce raw images that can be read by the VIO or other software.
(Back to report [5.2.2])

8.8 Common Files and Commands

8.8.1 Image decompressor

The *decompressor.py* file retrieves a stream of compressed images and republishes a topic containing the images in raw format. The vast majority of *ROS1* and *ROS2* packages in the visual-inertial domain work with raw image topics because they can be used directly. I'd like to point out that this code was written by AI.

You can consult the code on the drive : here .

8.8.2 ROS2-to-ROS1 bag

This is done simply with the command

```
python3 -m rosbags.convert --src UKF17 --dst UKF_ros1.bag
```

In this example, the *UKF17* rosbag is in *ROS2*, while *UKF_ros1.bag* is not a rosbag but a simple file. Once created, you'll need to place it inside the *UKF17* rosbag and call it directly by its name (i.e., *UKF_ros1.bag*) in commands that require *ROS1*-compatible.

8.8.3 Tf Tree

The TF tree of our overall ROS2 platform architecture is available by clicking here. This was obtained from the following command:

```
ros2 run tf2_tools view_frames
```

8.9 OptiTrack Tutorial

To use the *OptiTrack Motive* motion capture system, you must:

- Connect the router to the cameras, then connect the Ethernet cable from the router to the PC.
- Configure the VRPN Client, which is a module that connects to the OptiTrack motion capture server via the *VRPN* network protocol to receive real-time data on the position, orientation, velocity, and acceleration of the tracked objects. To do this, the OptiTrack host PC (Windows), the Jetson (Ubuntu), and the PC connected to the Jetson via SSH (Ubuntu) must all be on the same Wi-Fi network. Next, retrieve the IPv4 address of the Windows PC and its publishing port. Time synchronization of *OptiTrack* is disabled like this the recorded rosbag2 dataset will be synchronized with the ground truth (see 1.2) ; even if this can this will be handled by *evo* ([5]) during result processing.

```
'server': '192.168.42.128',
'port': 3883,
'update_freq': 100.0,
'frame_id': 'map',
'use_sim_time': False,
```

- Create the *RigidBody* by attaching infrared markers to the platform, which are then selected in the software to create a rigid body with an origin and a Cartesian coordinate system. Next, adjust the orientation and position of the coordinate system.
- Subscribe to the topic */RigidBody_004/pose* or *RigidBody_004*, which is the name of the body created in the software.

The configuration code for the ground truth topic is available in *system.launch.py*, which is the Python launch script for the entire system. You can view it here. Here is an overview of the software 8.7:

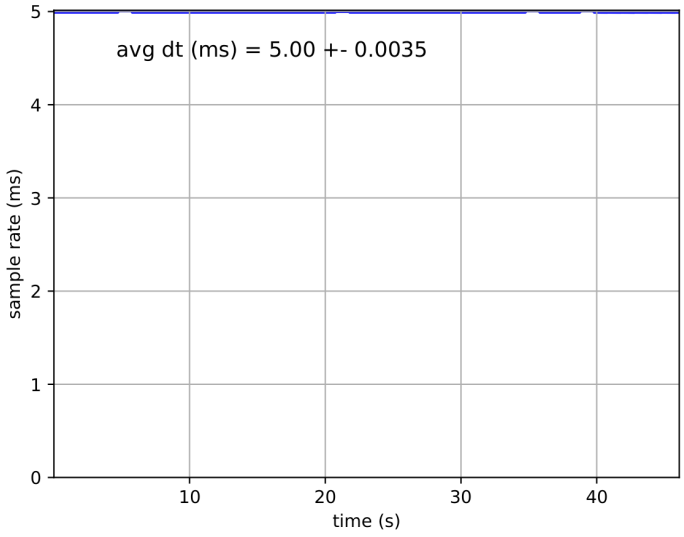


Figure 8.7: Visiualisation of the RigidBody on OptiTrack Motive

8.10 Package Evo Tutorial

The *evo* [5] package allows us to align the odometry topics as accurately as possible. The plots can be displayed on a PC or on a local web server. They are interactive and allow for a detailed analysis of trajectories and localization errors.

For the local web server:

```
source evo_env/bin/activate
rerun --serve-web     # terminal 1
evo_traj bag2 UKF20 /RigidBody_004/pose /odometry/filtered_ukf /ov_msckf/odomimu  --ref
   /RigidBody_004/pose   -a --rerun    #terminal 2 -> overlays the trajectories
```

For the interactive image:

```
evo_traj bag2 UKF20 /RigidBody_004/pose /ov_msckf/odomimu /odometry/filtered_ukf  --ref
   /RigidBody_004/pose  -a -p --plot_mode xyz    #overlays the trajectories

evo_ape bag2 UKF20  /RigidBody_004/pose /ov_msckf/odomimu  -a -p --plot_mode xyz  #
   Analyze the error with overlays the trajectories
```

In addition, when starting up, the odometry data published by the UKF/EKF fluctuates significantly in a fraction of a second. This can shift the starting point of the trajectory. To correct this position error, I use the command:

```
python3 crop_bag.py UKF20 UKF20_c 19
```

Where *crop_bag.py* is the Python file that will crop our rosbag, *UKF20* is the modified rosbag, *UKF20_c* is the new rosbag, and *19* is the number of seconds removed from the original rosbag. You can view the Python script by clicking here. Here is an example of how it appears on the web server 8.8:

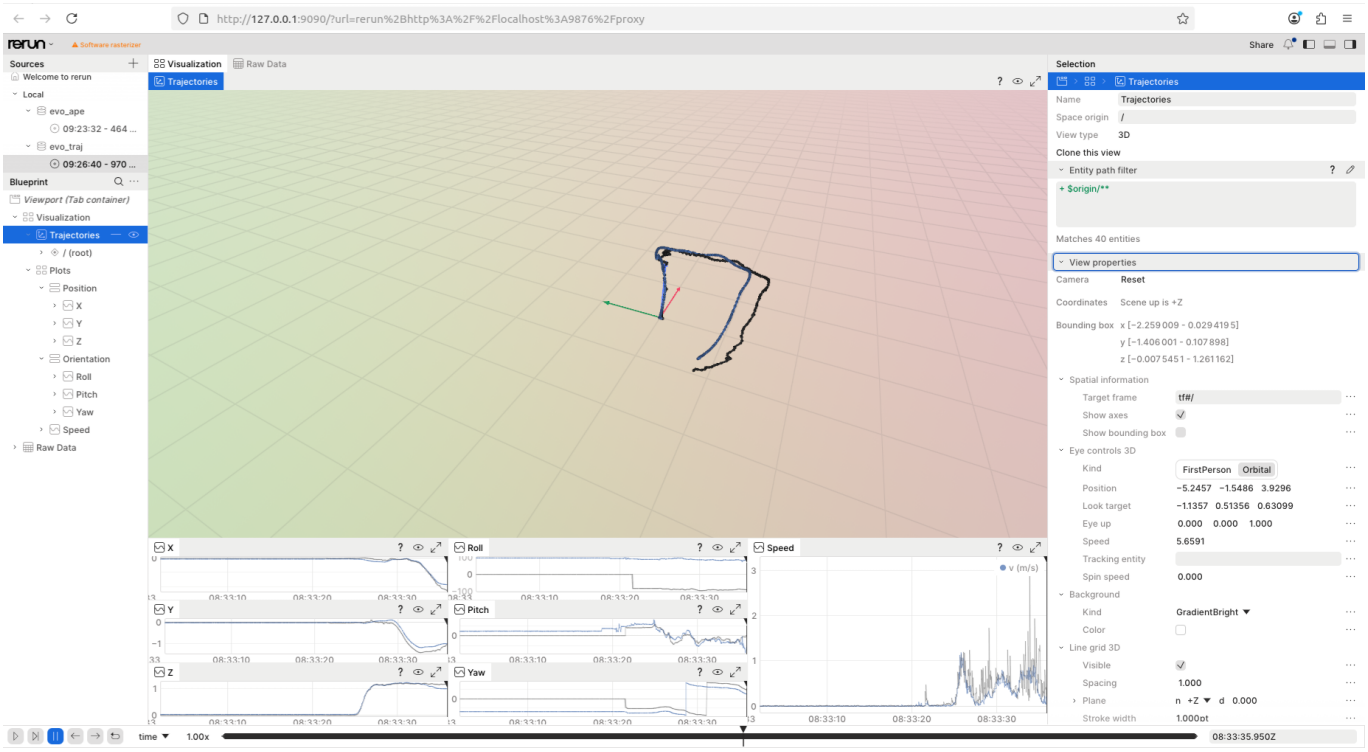


Figure 8.8: Evo Comparison Between OptiTrack and UKF

The velocities reported by OptiTrack are incorrect. Therefore, we will not include velocities in our analyses. For the orientation analysis, use the evo web server. A graphical analysis with numerical values is not necessary because the initial orientation between the rigidbody and the base_link is too different. We will therefore analyze the consistency of the coordinate system's orientation with the test performed.

To display the coordinate systems, expand the `Trajectories`, `root` pane and select the name of an odometry topic, such as that of the rigidbody, the VIO, or the UKF. Next, click on the fourth tab to the right of `Share` in the upper-right corner of the screen; this will open a drop-down panel on the right side of the web server. Then select the tab to display the `Frames` and change the axis scale from 0 to 0.3.

You will then be able to view the orientation of the coordinate system over time by playing the video in the lower-left corner of the page.

8.11 Data Acquisition Tutorial

In this section, you will find the steps to follow to retrieve the data generated by the ROS2 architecture.

First, you must *build* the workspace on the computer and then send it to the Jetson. From Terminal 1 on the computer, run the following command:

```
cd ~/sensor_platform_ws
colcon build
rsync -avzc --delete --exclude 'build' --exclude 'install' --exclude 'log' ~/
   sensor_platform_ws/src/ darts@192.168.42.124:~/sensor_platform/src/
```

Note: The Jetson's IP address depends on the Wi-Fi network (see 8.5.1).

Open two new terminals on the PC, then connect to the Jetson (the password is "*crta*"):

```
ssh darts@192.168.42.124
```

In Terminals 2 and 3, connected via SSH, navigate to the correct folder and connect to the ROS domain for our architecture. In Terminal 2, start *Holybro* and launch the global system:

```
cd sensor_platform_ws
export ROS_DOMAIN\_ID=3
sudo slcand -o -s8 -t hw -S 3000000 /dev/ttyACM0 slcan0
sudo ip link set up slcan0
source ~/sensor_platform_ws/install/setup.bash
ros2 launch platform_bringup system.launch.py
```

In Terminal 3, run the terminal so you can execute the rosbag generation file. Make sure you've included the topics you want to record in this script.

```
cd sensor_platform_ws
export ROS_DOMAIN\_ID=3
ros2 topic list
source install/setup.bash
ros2 launch platform_bringup record_dataset.launch.py
```

Once data acquisition is complete, press *Ctrl+C* once in terminals 2 and 3. Then, in terminal 1 associated with the PC, push the rosbag to the (example) *slam_ws* folder on the PC, naming it *UKF20*.

```
rsync -avz darts@192.168.42.124:~/sensor_platform_ws/UKF1/ ~/slam_ws/UKF20
```

Unless you set up a computer monitor on the platform, you'll need to send the rosbags to the PC because the Jetson doesn't have any graphics capabilities.